Introduction to Java EE

Enterprise Edition — Study Notes

March 25, 2026

Copyright Notice

© 2026 Amitendu Bikash Dhusiya

All rights reserved.

You may use, copy, modify, and share this material for any purpose.
Please acknowledge the author: **Amitendu Bikash Dhusiya**.

Contents

1 Introduction to Java EE **7**
1.1 Overview of Java EE Architecture . . . 7
1.1.1 Java EE Containers . . . 7
1.1.2 Core Java EE API Summary . . . 8
1.2 Difference Between Java SE and Java EE . . . 8
1.3 Role of Technologies in Web Applications . . . 9
1.4 JDBC — Java Database Connectivity . . . 10
1.5 JSP — JavaServer Pages . . . 10
1.5.1 JSP Lifecycle . . . 10
1.5.2 JSP Syntax Elements . . . 10
1.5.3 JSP Implicit Objects . . . 11
1.6 EL — Expression Language . . . 11
1.6.1 Why EL? . . . 12
1.6.2 EL Syntax . . . 12
1.6.3 EL Scope Resolution Order . . . 12
1.6.4 EL Implicit Objects . . . 12
1.6.5 EL with JSTL — Code Example . . . 13
1.7 Servlets — Java Servlets . . . 14
1.7.1 Servlet Lifecycle . . . 14
1.7.2 HTTP Method Mapping . . . 14
1.7.3 Servlet Code Example . . . 15
1.7.4 Forward vs. Redirect . . . 15
1.8 How JDBC, JSP, EL, and Servlets Work Together . . . 16

2 JDBC — Java Database Connectivity **17**
2.1 Introduction to JDBC . . . 17
2.2 JDBC Drivers and Architecture . . . 17
2.2.1 JDBC Driver Types . . . 17
2.2.2 Core JDBC Interfaces and Classes (`java.sql`) . . . 18
2.3 Connecting to Databases . . . 18
2.3.1 Common JDBC Connection URLs . . . 19
2.3.2 Connection Code Example . . . 19
2.3.3 Connection Pooling . . . 19
2.4 Executing SQL Queries (SELECT, INSERT, UPDATE, DELETE) . . . 20
2.4.1 Full CRUD Code Example . . . 20
2.5 Use of Statement and PreparedStatement . . . 21
2.5.1 PreparedStatement Setter Methods . . . 22
2.5.2 CallableStatement (Stored Procedures) . . . 22

2.6 ResultSet and Metadata . . . 23
2.6.1 ResultSet Types . . . 23
2.6.2 Common ResultSet Methods . . . 23
2.6.3 ResultSetMetaData . . . 24
2.6.4 DatabaseMetaData . . . 24
2.7 Transaction Management . . . 24
2.7.1 ACID Properties . . . 25
2.7.2 Auto-Commit Mode . . . 25
2.8 Commit, Rollback, and Savepoints . . . 25
2.8.1 Savepoint Concept . . . 26
2.8.2 Full Transaction Example with Savepoint . . . 26
2.9 Batch Processing . . . 27
2.9.1 Batch vs. Individual Execution . . . 27
2.9.2 Batch Processing API . . . 27
2.9.3 Batch Insert Code Example . . . 28
2.9.4 Periodic Flush for Very Large Batches . . . 29
2.10 Module 2 Summary . . . 29

3 Java Server Pages (JSP) 31
3.1 Introduction to JSP . . . 31
3.2 JSP Lifecycle . . . 31
3.3 JSP Syntax and Directives . . . 33
3.4 Scriptlet, Expression and Declaration . . . 33
3.4.1 Scriptlet `<% ... %>` . . . 33
3.4.2 Expression `<%= ... %>` . . . 33
3.4.3 Declaration `<%! ... %>` . . . 33
3.5 JSP Implicit Objects . . . 34
3.6 JSP Directives . . . 36
3.6.1 Page Directive — `<%@ page %>` . . . 36
3.6.2 Include Directive — `<%@ include %>` . . . 37
3.6.3 Taglib Directive — `<%@ taglib %>` . . . 37
3.7 JSP Action Elements . . . 38
3.7.1 `<jsp:include>` . . . 38
3.7.2 `<jsp:forward>` . . . 38
3.8 Java Beans in JSP . . . 39
3.8.1 Using JavaBeans with JSP Action Tags . . . 40
3.9 Introduction to JSP Expression Language (EL) . . . 40
3.9.1 EL vs. Scriptlet . . . 41
3.9.2 EL Implicit Objects . . . 41
3.9.3 EL Operators . . . 41

3.10 Introduction to JSTL Core Tags . . . 42
3.10.1 `<c:out>` — Output a Value . . . 42
3.10.2 `<c:set>` — Set a Variable . . . 43
3.10.3 `<c:remove>` — Remove a Variable . . . 43
3.10.4 `<c:if>` — Conditional Rendering . . . 43
3.10.5 `<c:choose>`, `<c:when>`, `<c:otherwise>` — if-else Chain . . . 43
3.10.6 `<c:forEach>` — Iteration . . . 44
3.10.7 `<c:forTokens>` — Tokenize a String . . . 44
3.10.8 `<c:import>` — Import a Resource . . . 45
3.10.9 `<c:url>` — URL Encoding . . . 45
3.10.10 `<c:redirect>` — Redirect the Browser . . . 45
3.11 JSTL Functions . . . 46
3.12 Custom Tag Library . . . 48
3.12.1 Components of a Custom Tag . . . 48
3.12.2 Step 1 — Create the Tag Handler Class . . . 48
3.12.3 Step 2 — Create the TLD File . . . 49
3.12.4 Step 3 — Use the Custom Tag in a JSP . . . 49
3.12.5 Custom Tag with Body Content . . . 50
3.13 Module 3 Summary . . . 51

4 Servlets **52**
4.1 Introduction to Servlets . . . 52
4.2 Servlet Life Cycle . . . 53
4.3 Handling HTTP Requests and Responses . . . 53
4.3.1 HttpServletRequest . . . 54
4.3.2 HttpServletResponse . . . 54
4.3.3 RequestDispatcher — Forward vs. Redirect . . . 55
4.4 Servlet Deployment Descriptor (`web.xml`) . . . 55
4.5 ServletContext and ServletConfig . . . 57
4.5.1 ServletConfig . . . 57
4.5.2 ServletContext . . . 57
4.6 Session Management and Cookies . . . 58
4.6.1 Cookies . . . 58
4.6.2 HttpSession . . . 59
4.7 Servlet Chaining and Filters . . . 59
4.7.1 Servlet Chaining . . . 59
4.7.2 Servlet Filters . . . 60
4.8 File Upload and Download in Servlet . . . 61
4.8.1 File Upload . . . 61
4.8.2 File Download . . . 62

4.9 Module 4 Summary . . . 64

5 JSP and Servlet — Integration **65**

5.1 Combining JSP and Servlets . . . 65

5.2 Model-View-Controller (MVC) Architecture . . . 67

5.2.1 MVC Request Flow . . . 67

5.2.2 Complete MVC Example — Student List . . . 67

5.2.3 Front Controller Pattern . . . 69

5.3 Forwarding Requests Between JSP and Servlet . . . 70

5.3.1 `RequestDispatcher.forward()` — Servlet to JSP . . . 71

5.3.2 `RequestDispatcher.include()` — Embedding Fragments . . . 71

5.3.3 JSP forwarding back to a Servlet . . . 71

5.3.4 Passing Data with Request Attributes . . . 72

5.3.5 Post-Redirect-Get (PRG) Pattern . . . 73

5.4 Module 5 Summary . . . 74

6 Hibernate Framework **75**

6.1 Advantages of ORM over JDBC . . . 75

6.1.1 Limitations of Plain JDBC . . . 75

6.1.2 How ORM / Hibernate Solves These . . . 75

6.2 Hibernate Architecture and Core Components . . . 77

6.2.1 Layered Architecture Overview . . . 77

6.2.2 Core Components . . . 77

6.2.3 Object / Entity States in Hibernate . . . 78

6.2.4 First-Level Cache (Session Cache) . . . 79

6.3 Setting Up Hibernate in a Java Application . . . 79

6.3.1 Project Dependencies (Maven) . . . 79

6.3.2 HibernateUtil — SessionFactory Helper . . . 80

6.4 Hibernate Configuration . . . 80

6.4.1 XML Configuration — `hibernate.cfg.xml` . . . 80

6.5 Mapping Java Classes to Database Tables . . . 82

6.5.1 Annotation-Based Mapping (Recommended) . . . 82

6.5.2 XML-Based Mapping — `Student.hbm.xml` (Legacy) . . . 84

6.6 Basic CRUD Operations . . . 85

6.6.1 Save (Create) . . . 85

6.6.2 Retrieve (Read) . . . 86

6.6.3 Update . . . 87

6.6.4 Delete . . . 88

6.6.5 saveOrUpdate() and merge() . . . 89

6.7 Hibernate Query Language (HQL) . . . 89

6.7.1 Basic HQL Syntax 90
6.7.2 Named Parameters in HQL 90
6.7.3 Pagination with HQL 91
6.7.4 Aggregate Functions in HQL 91
6.7.5 HQL UPDATE and DELETE (Bulk Operations) 92
6.7.6 Named Queries 93
6.7.7 Native SQL Queries 93
6.8 Module 6 Summary 94

MODULE 1 — Introduction to Java EE

What is Java EE?

Java EE (Java Platform, Enterprise Edition), rebranded as **Jakarta EE** since 2019, is a collection of specifications and APIs that extend Java SE. It provides a standardised runtime environment for building large-scale, multi-tiered, reliable, and secure *enterprise web applications*.

1.1 Overview of Java EE Architecture

Java EE follows a **multi-tier (n-tier) architecture**, separating concerns into distinct layers so each can evolve independently.

Java EE Multi-Tier Architecture

Tier	Responsibility
Client Tier	Browser, desktop GUI, or mobile app. Sends requests and displays responses.
Web Tier	Handles HTTP. Contains Servlets and JSPs inside the *Web Container*.
Business Tier	Holds business logic: EJBs, CDI beans, managed beans.
Data/EIS Tier	Relational databases or legacy systems. Accessed via JDBC or JPA.

1.1.1 Java EE Containers

A **container** is the runtime environment within the application server. It manages component lifecycles and provides services such as security, transactions, and naming.

Types of Java EE Containers

- **Web Container (Servlet Container):** Manages Servlets, JSPs, and filters. Example: Apache Tomcat, Jetty.
- **EJB Container:** Manages Enterprise JavaBeans with declarative transactions and security.
- **Application Client Container:** For standalone Java client applications.

1.1.2 Core Java EE API Summary

Key Java EE Technologies

Technology	Purpose
`Servlet API`	Low-level HTTP request/response handling (Controller)
`JSP / EL`	Dynamic HTML generation; Expression Language for views (View)
`JDBC`	Relational database connectivity (Model/Data)
`JPA`	Object-Relational Mapping (ORM)
`EJB`	Distributed, transactional business components
`JMS`	Asynchronous messaging
`JAX-RS`	RESTful Web Services
`CDI`	Contexts and Dependency Injection

1.2 Difference Between Java SE and Java EE

Core Distinction

- **Java SE** provides the *core language* and foundational libraries: collections, I/O, threading, networking.
- **Java EE** *extends* Java SE with enterprise APIs for web components, distributed computing, transactions, messaging, and persistence.

Java SE vs. Java EE — Comparison

Feature	Java SE	Java EE
Full Name	Standard Edition	Enterprise Edition
Target	Desktop, console, small apps	Large-scale enterprise/web apps
Base APIs	Core Java (`java.lang`, `java.util`...)	Java SE + enterprise APIs
Runtime	JVM / JDK	Application Server (Tomcat, WildFly...)
Web Support	None natively	Servlets, JSP, EL, JAX-RS
Database	Basic JDBC	JDBC + JPA (full ORM)
Transactions	Manual	Container-managed (CMT)
Messaging	None	Java Message Service (JMS)
Scalability	Limited	Built for horizontal scaling
Example	CLI tool, calculator	Online banking, e-commerce

Important Note

Java EE is a *superset* of Java SE — it does not replace it. The JDK must be installed before any Java EE application server can operate.

1.3 Role of Technologies in Web Applications

The technologies below collectively implement the **Model-View-Controller (MVC)** pattern:

MVC Mapping in Java EE

- **Model** ⟶ JDBC (data access / persistence layer)
- **View** ⟶ JSP + EL (presentation / UI layer)
- **Controller** ⟶ Servlet (request handling / routing)

1.4 JDBC — Java Database Connectivity

Definition: JDBC

JDBC (Java Database Connectivity) is a Java API in `java.sql` and `javax.sql` that provides a *standard, vendor-independent* interface for connecting to relational databases, executing SQL, and retrieving results. Full JDBC coverage is in **Module 2**.

1.5 JSP — JavaServer Pages

Definition: JSP

JSP (JavaServer Pages) is a server-side technology that allows developers to embed Java code within HTML using special tags to produce *dynamically generated web pages*. JSP files are translated and compiled into Servlets automatically by the web container.

1.5.1 JSP Lifecycle

JSP Processing Lifecycle

1. **Translation:** `.jsp` → Java Servlet source (once, or on change).
2. **Compilation:** Java source → `.class` bytecode.
3. **Loading & Instantiation:** Class loaded; Servlet object created.
4. **Initialization:** `jspInit()` called once.
5. **Request Processing:** `_jspService()` called per request.
6. **Destruction:** `jspDestroy()` called on container shutdown.

1.5.2 JSP Syntax Elements

JSP Tag Reference

Syntax	Purpose	When to Use
`<% ... %>`	Scriptlet — Java code block	Loops, conditionals
`<%= ... %>`	Expression — prints value	Display a variable
`<%! ... %>`	Declaration — fields/methods	Helper methods
`<%@ page %>`	Page directive	imports, contentType
`<%@ include %>`	Include directive	Header, footer
`<%@ taglib %>`	Tag library	JSTL usage
`<jsp:useBean>`	Instantiate/locate a JavaBean	Bean in scope
`<jsp:getProperty>`	Read a bean property	Display bean data
`<jsp:forward>`	Forward to another resource	Routing

1.5.3 JSP Implicit Objects

Nine JSP Implicit Objects (auto-provided, no declaration needed)

Object	Type and Description
`request`	`HttpServletRequest` — incoming HTTP request data
`response`	`HttpServletResponse` — response to client
`out`	`JspWriter` — output stream to the browser
`session`	`HttpSession` — user session data
`application`	`ServletContext` — application-wide shared data
`config`	`ServletConfig` — JSP configuration info
`pageContext`	`PageContext` — page-level attributes and access
`page`	`Object` — reference to the current JSP (like `this`)
`exception`	`Throwable` — available only in error pages

1.6 EL — Expression Language

Definition: Expression Language (EL)

EL (Expression Language) is a *simplified scripting language* built into the JSP specification that provides read/write access to Java objects stored in various *scopes* (page, request, session, application) without writing raw Java code inside the JSP. EL expressions use the syntax `${expression}`.

1.6.1 Why EL?

Raw Java scriptlets inside JSP (`<% %>`) violate MVC separation by mixing business logic with the view. EL replaces scriptlets with clean, readable, and *injection-safe* expressions directly in HTML markup, making templates readable even to non-Java developers.

1.6.2 EL Syntax

EL Syntax Reference

Syntax	Description
`${name}`	Read attribute `name` from any scope
`${user.firstName}`	Dot operator — calls `user.getFirstName()`
`${map["key"]}`	Bracket operator — map entry or array index
`${a + b}`	Arithmetic: `+`, `-`, `*`, `/` (or `div`), `%` (or `mod`)
`${a == b}`	Relational: `==`, `!=`, `<`, `>`, `<=`, `>=`
`${a && b}`	Logical: `&&` (and), `||` (or), `!` (not)
`${empty list}`	`empty` — true if null, empty string, or empty collection
`${a ? b : c}`	Ternary / conditional operator
`#{bean.value}`	Deferred evaluation (JSF; not standard JSP)

1.6.3 EL Scope Resolution Order

When you write `${username}`, EL searches scopes in this fixed order:

EL Implicit Scope Search Order

pageScope → requestScope → sessionScope → applicationScope

You can also target a specific scope explicitly:

`${sessionScope.username}` — reads *only* from the session scope.

1.6.4 EL Implicit Objects

EL Implicit Objects

EL Object	Provides Access To
`pageScope`	Attributes in page scope
`requestScope`	Attributes in request scope
`sessionScope`	Attributes in session scope
`applicationScope`	Attributes in application scope
`param`	Single request parameter (`${param.name}`)
`paramValues`	Multi-value request parameter (array)
`header`	Single HTTP request header
`headerValues`	Multi-value HTTP request header
`cookie`	Cookie values by name
`initParam`	Context init parameters from `web.xml`
`pageContext`	Full access to the JSP `PageContext` object

1.6.5 EL with JSTL — Code Example

JSP + EL + JSTL (students.jsp)

```jsp
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head><title>Student List</title></head>
<body>

  <%-- EL: read session attribute set by the Servlet --%>
  <h2>Welcome, ${sessionScope.loggedInUser}!</h2>

  <%-- EL with empty operator --%>
  <c:if test="${empty studentList}">
    <p>No students found.</p>
  </c:if>

  <%-- EL inside JSTL forEach; dot notation calls getter --%>
  <c:forEach var="s" items="${studentList}" varStatus="loop">
    <p>
      ${loop.index + 1}.
      ${s.name} (${s.dept})
      -- ${s.gpa > 3.5 ? "Distinction" : "Pass"}
```

```
    </p>
  </c:forEach>

  <%-- EL: read request parameter directly --%>
  <p>Search term: ${param.query}</p>

</body>
</html>
```

EL vs. Scriptlets — Best Practice

Always prefer EL + JSTL over scriptlets. EL is:

- **Cleaner** — no Java code embedded in HTML.
- **Safer** — EL automatically escapes output, reducing XSS risk.
- **Maintainable** — UI designers can read `${user.name}` without knowing Java.

1.7 Servlets — Java Servlets

Definition: Servlet

A **Servlet** is a Java class that runs inside a *Servlet Container* and handles **HTTP requests**. It receives client requests, orchestrates business logic and data access (JDBC), and sends back HTTP responses — typically by forwarding to a JSP/EL view.

1.7.1 Servlet Lifecycle

Servlet Lifecycle — Container-Managed

1. **Loading & Instantiation:** One instance per Servlet class created.
2. **`init()`:** Called *once*; one-time setup.
3. **`service()`:** Called per request; dispatches to `doGet()`, `doPost()`, etc.
4. **`destroy()`:** Called *once* on shutdown; cleanup.

1.7.2 HTTP Method Mapping

Servlet HTTP Method Handlers

Method	HTTP Verb	Typical Use
`doGet()`	GET	Retrieve data, display pages, search
`doPost()`	POST	Submit forms, create resources
`doPut()`	PUT	Update a resource (REST)
`doDelete()`	DELETE	Delete a resource (REST)

1.7.3 Servlet Code Example

Servlet -- Student Controller

```java
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.*;
import java.util.List;

@WebServlet("/students")   // URL: http://host/app/students
public class StudentServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req,
                         HttpServletResponse res)
            throws ServletException, IOException {

        // 1. Call data layer (JDBC DAO)
        List<Student> students = StudentDAO.findAll();

        // 2. Place data in request scope for EL in JSP
        req.setAttribute("studentList", students);

        // 3. Forward to JSP View
        req.getRequestDispatcher("/WEB-INF/students.jsp")
           .forward(req, res);
    }
}
```

1.7.4 Forward vs. Redirect

Forward vs. Redirect

	Forward (`RequestDispatcher`)	**Redirect** (`sendRedirect`)
Processing	Server-side only	Client issues a new request
Browser URL	Unchanged	Updated to new URL
Request attributes	Preserved	Lost
Speed	Faster (no round-trip)	Slower (extra HTTP round-trip)
Use case	Pass data to JSP view	Post-Redirect-Get pattern

1.8 How JDBC, JSP, EL, and Servlets Work Together

End-to-End MVC Request Flow

1. **Browser** sends HTTP request to `/students`.
2. **Servlet Container** routes it to `StudentServlet.doGet()`.
3. **Servlet** validates parameters; calls **JDBC** DAO.
4. **JDBC** queries the database; returns a list of objects.
5. **Servlet** stores the list as a `request` attribute.
6. **Servlet** forwards to `students.jsp`.
7. **JSP + EL** renders data using `${studentList}` expressions.
8. **Browser** receives and displays the final HTML.

Module 1 Key Takeaway

- **Servlet** = Controller; **JSP + EL** = View; **JDBC** = Model.
- EL bridges the Servlet (stores data as attributes) and JSP (displays it).
- This MVC pattern is the foundation of all modern Java web frameworks (Spring MVC, Jakarta Faces).

MODULE 2 — JDBC — Java Database Connectivity

2.1 Introduction to JDBC

Definition: JDBC

JDBC (Java Database Connectivity) is a standard Java API defined in the packages `java.sql` and `javax.sql`. It provides a *uniform interface* for Java applications to interact with any relational database (MySQL, PostgreSQL, Oracle, SQL Server, SQLite, etc.) using **SQL** — regardless of the underlying database vendor.

Key benefits of JDBC:

- **Database Independence:** Write once; swap the driver to change databases.
- **Standard API:** Consistent classes and interfaces across all vendors.
- **Full SQL Support:** DDL, DML, stored procedures, and transactions.
- **Integration Base:** Foundation for JPA, Hibernate, and Spring JDBC.

2.2 JDBC Drivers and Architecture

JDBC Architecture (Two Layers)

- **JDBC API Layer:** Application-facing; provides interfaces `Connection`, `Statement`, `ResultSet`, etc.
- **JDBC Driver Layer:** Vendor-provided; implements the API interfaces to communicate with the specific database engine.

2.2.1 JDBC Driver Types

Four Types of JDBC Drivers

Type	Name	How It Works	Status
1	JDBC-ODBC Bridge	Bridges to ODBC; uses native ODBC driver	Obsolete
2	Native API Driver	Calls vendor's native C/C++ libraries via JNI	Rare
3	Network Protocol	Uses middleware server; pure Java	Uncommon
4	Thin (Pure Java)	Direct DB protocol over TCP/IP; pure Java JAR	**Standard**

Type 4 is Universal Today

Type 4 (Pure Java / Thin Driver) requires only one JAR file (e.g., `mysql-connector-java.jar`) and communicates natively with the database over TCP/IP — no native libraries or middleware needed.

2.2.2 Core JDBC Interfaces and Classes (`java.sql`)

JDBC API Components

Component	Role
`DriverManager`	Manages drivers; creates `Connection` objects
`Connection`	Open session with the database
`Statement`	Executes static SQL; SQL-injection risk
`PreparedStatement`	Pre-compiled parameterised SQL; injection-safe
`CallableStatement`	Executes stored procedures
`ResultSet`	Cursor over rows returned by a query
`ResultSetMetaData`	Metadata about columns in a `ResultSet`
`DatabaseMetaData`	Metadata about the database itself
`SQLException`	Checked exception for all database errors
`Savepoint`	Named point within a transaction for partial rollback

2.3 Connecting to Databases

JDBC Connection Steps

1. Add the **Driver JAR** to the classpath (e.g., `mysql-connector.jar`).
2. JDBC 4+ **auto-loads** the driver via Java Service Loader (no `Class.forName` needed).
3. Build the **Connection URL**: `jdbc:<subprotocol>://<host>:<port>/<database>`
4. Call `DriverManager.getConnection(url, user, password)`.
5. Always **close** the connection — use try-with-resources.

2.3.1 Common JDBC Connection URLs

JDBC URL Formats by Database

Database	JDBC URL Format
MySQL	`jdbc:mysql://localhost:3306/dbname`
PostgreSQL	`jdbc:postgresql://localhost:5432/dbname`
Oracle	`jdbc:oracle:thin:@localhost:1521:XE`
SQL Server	`jdbc:sqlserver://localhost:1433;databaseName=db`
SQLite	`jdbc:sqlite:/path/to/file.db`
H2 (memory)	`jdbc:h2:mem:testdb`

2.3.2 Connection Code Example

Establishing a JDBC Connection (try-with-resources)

```java
import java.sql.*;

public class DBConnect {

    static final String URL  =
        "jdbc:mysql://localhost:3306/university?useSSL=false";
    static final String USER = "root";
    static final String PASS = "password";

    public static void main(String[] args) {

        // try-with-resources: Connection auto-closed on block exit
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {

            System.out.println("Connected to: "
                + con.getMetaData().getDatabaseProductName());

        } catch (SQLException e) {
            System.err.println("Connection failed: " + e.getMessage());
        }
    }
}
```

2.3.3 Connection Pooling

Opening a new `Connection` per HTTP request is expensive (TCP handshake + authentication). **Connection Pooling** maintains pre-opened reusable connections:

Popular Connection Pool Libraries

- **HikariCP** — fastest, minimal footprint (default in Spring Boot).
- **Apache DBCP2** — Apache Commons pool.
- **c3p0** — older, highly configurable.
- **App Server Pool** — JNDI `DataSource` in Java EE containers.

2.4 Executing SQL Queries (SELECT, INSERT, UPDATE, DELETE)

JDBC Execution Methods

Method	Description	Returns
`executeQuery(sql)`	Executes SELECT	`ResultSet`
`executeUpdate(sql)`	Executes INSERT/UPDATE/DELETE/DDL	`int` (rows affected)
`execute(sql)`	Any SQL; used for dynamic cases	`boolean`
`executeBatch()`	Runs all batched statements	`int[]`

2.4.1 Full CRUD Code Example

JDBC CRUD Operations

```java
import java.sql.*;

public class StudentDAO {
    static final String URL  = "jdbc:mysql://localhost:3306/university";
    static final String USER = "root", PASS = "password";

    // SELECT
    public void selectAll() throws SQLException {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery(
                 "SELECT id, name, dept FROM students")) {
            while (rs.next()) {
                System.out.printf("%-4d %-20s %s%n",
                    rs.getInt("id"),
                    rs.getString("name"),
                    rs.getString("dept"));
            }
        }
    }

```

```java
    // INSERT
    public void insert(String name, String dept) throws SQLException {
        String sql = "INSERT INTO students (name, dept) VALUES (?, ?)";
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, name);
            ps.setString(2, dept);
            System.out.println(ps.executeUpdate() + " row(s) inserted.");
        }
    }

    // UPDATE
    public void update(int id, String newDept) throws SQLException {
        String sql = "UPDATE students SET dept = ? WHERE id = ?";
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, newDept);
            ps.setInt(2, id);
            ps.executeUpdate();
        }
    }

    // DELETE
    public void delete(int id) throws SQLException {
        String sql = "DELETE FROM students WHERE id = ?";
        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```

2.5 Use of Statement and PreparedStatement

Statement vs. PreparedStatement

Feature	Statement	PreparedStatement
SQL Type	Static (no parameters)	Parameterised (`?` placeholders)
SQL Injection	**Vulnerable**	**Safe** (parameters escaped)
Compilation	Every execution	Pre-compiled once; reused
Performance	Slower for repeated use	Faster for repeated calls
Use Case	DDL, one-off ad-hoc SQL	All DML with user input

Security: Always Use PreparedStatement with User Input

Concatenating user input into SQL strings (`"... WHERE name = '" + input + "'"`) is **SQL Injection vulnerable**. Input like `' OR '1'='1` would expose all rows. `PreparedStatement` automatically escapes all parameters.

2.5.1 PreparedStatement Setter Methods

Common `PreparedStatement` Setters

Method	Sets
`setInt(n, val)`	Integer at placeholder `n`
`setString(n, val)`	String value
`setDouble(n, val)`	Double / floating-point value
`setBoolean(n, val)`	Boolean value
`setDate(n, val)`	`java.sql.Date`
`setTimestamp(n, val)`	`java.sql.Timestamp`
`setNull(n, type)`	Explicit SQL NULL
`setObject(n, val)`	Any Java object (auto-mapped)

2.5.2 CallableStatement (Stored Procedures)

CallableStatement -- Stored Procedure

```java
// Stored proc: CALL getStudentsByDept(IN dept VARCHAR, OUT cnt INT)
CallableStatement cs =
    con.prepareCall("{call getStudentsByDept(?, ?)}");
cs.setString(1, "Computer Science");        // IN parameter
cs.registerOutParameter(2, Types.INTEGER); // OUT parameter
cs.execute();
int count = cs.getInt(2);                   // Read OUT value
```

```
System.out.println("Students: " + count);
```

2.6 ResultSet and Metadata

Definition: ResultSet

A `ResultSet` is a table of data (cursor) returned by a SELECT query. The cursor starts *before* the first row; `next()` advances it row by row.

2.6.1 ResultSet Types

ResultSet Type and Concurrency Constants

Constant	Meaning
`TYPE_FORWARD_ONLY (default)`	Cursor moves forward only
`TYPE_SCROLL_INSENSITIVE`	Scrollable; does not reflect live DB changes
`TYPE_SCROLL_SENSITIVE`	Scrollable; reflects concurrent DB changes
`CONCUR_READ_ONLY (default)`	Cannot update rows via `ResultSet`
`CONCUR_UPDATABLE`	Can update rows via `ResultSet`

2.6.2 Common ResultSet Methods

ResultSet Navigation and Retrieval

Method	Description
`next()`	Advance to next row; returns `false` at end
`previous()`	Move to previous row (scrollable only)
`first() / last()`	Jump to first/last row (scrollable only)
`absolute(n)`	Jump to row `n` (scrollable only)
`getInt(col)`	Get integer value by column name or index
`getString(col)`	Get String value
`getDouble(col)`	Get double value
`getDate(col)`	Get `java.sql.Date`
`wasNull()`	Check if last retrieved column was SQL NULL

2.6.3 ResultSetMetaData

Definition: ResultSetMetaData

`ResultSetMetaData` provides information *about the columns* in a `ResultSet`: count, names, data types, sizes, and nullability. Used to write generic, schema-agnostic data access code.

ResultSetMetaData Example

```java
ResultSet rs   = stmt.executeQuery("SELECT * FROM students");
ResultSetMetaData meta = rs.getMetaData();

int cols = meta.getColumnCount();
System.out.println("Columns: " + cols);

for (int i = 1; i <= cols; i++) {
    System.out.println(meta.getColumnName(i)
        + " [" + meta.getColumnTypeName(i) + "]");
}

while (rs.next()) {
    for (int i = 1; i <= cols; i++)
        System.out.print(rs.getString(i) + "\t");
    System.out.println();
}
```

2.6.4 DatabaseMetaData

`DatabaseMetaData` provides information about the *database itself*: product name, version, supported SQL features, available tables, etc.

DatabaseMetaData Example

```java
DatabaseMetaData dbMeta = con.getMetaData();
System.out.println("DB      : " + dbMeta.getDatabaseProductName());
System.out.println("Version : " + dbMeta.getDatabaseProductVersion());
System.out.println("Driver  : " + dbMeta.getDriverName());
System.out.println("URL     : " + dbMeta.getURL());
```

2.7 Transaction Management

Definition: Transaction

A **transaction** is a sequence of SQL operations executed as a *single logical unit of work*. Either **all** operations succeed (*commit*) or **none** take effect (*rollback*), guaranteeing data integrity.

2.7.1 ACID Properties

ACID Properties

Property	Meaning
Atomicity	All-or-nothing: all steps commit or none do.
Consistency	DB transitions from one valid state to another.
Isolation	Concurrent transactions do not interfere with each other.
Durability	Committed changes persist even after a system crash.

2.7.2 Auto-Commit Mode

By default JDBC is in **auto-commit mode**: every SQL statement is immediately committed. For manual transaction control:

Disabling Auto-Commit

```
con.setAutoCommit(false);    // Begin manual transaction
// ... execute SQL ...
con.commit();                // Commit all changes
// or:
con.rollback();              // Undo all changes
con.setAutoCommit(true);     // Restore (important for pooled connections)
```

2.8 Commit, Rollback, and Savepoints

Transaction Control Methods

Method	Description
`con.setAutoCommit(false)`	Begin manual transaction
`con.commit()`	Permanently save all changes
`con.rollback()`	Undo all changes back to last commit
`con.setSavepoint("name")`	Mark a named point within a transaction
`con.rollback(savepoint)`	Roll back only to the named savepoint
`con.releaseSavepoint(sp)`	Remove a savepoint (free resources)
`con.setAutoCommit(true)`	Restore auto-commit after transaction

2.8.1 Savepoint Concept

A **Savepoint** is a named marker set within an active transaction. It allows *partial rollback*: you can undo operations done after the savepoint while keeping operations done before it.

Savepoint Usage Scenario

1. Start transaction (`setAutoCommit(false)`).
2. Execute Operation A successfully.
3. Set savepoint **SP1**.
4. Execute Operation B (may fail).
5. If B fails: `rollback(SP1)` — undoes B; A is preserved.
6. Commit A.

2.8.2 Full Transaction Example with Savepoint

Transaction: Commit, Rollback, Savepoint (Bank Transfer)

```java
Connection con = DriverManager.getConnection(URL, USER, PASS);
Savepoint sp = null;

try {
    con.setAutoCommit(false);                // Step 1: begin transaction

    // Step 2: Debit operation
    PreparedStatement ps1 = con.prepareStatement(
        "UPDATE accounts SET balance = balance - ? WHERE id = ?");
    ps1.setDouble(1, 500.00);
    ps1.setInt(2, 101);                      // Account 101 debited
    ps1.executeUpdate();

    // Step 3: Savepoint after successful debit
    sp = con.setSavepoint("after_debit");

    // Step 4: Credit operation
    PreparedStatement ps2 = con.prepareStatement(
        "UPDATE accounts SET balance = balance + ? WHERE id = ?");
    ps2.setDouble(1, 500.00);
    ps2.setInt(2, 202);                      // Account 202 credited
    ps2.executeUpdate();

    con.commit();                            // Step 5: commit both
    System.out.println("Transfer successful.");

} catch (SQLException e) {
    if (sp != null) {
        con.rollback(sp);                    // Undo only credit; keep debit
        System.out.println("Partial rollback to savepoint.");
```

```
        con.commit();                          // Commit the debit only
    } else {
        con.rollback();                        // Full rollback
        System.out.println("Full rollback.");
    }
} finally {
    con.setAutoCommit(true);                   // Always restore
    con.close();
}
```

Transaction Best Practices

- Always wrap multi-step DML in **try-catch-finally** with rollback in catch.
- Keep transactions **as short as possible** to reduce DB lock contention.
- Use **Savepoints** to partially undo large transactions without losing all work.
- Always call `setAutoCommit(true)` in `finally` when reusing pooled connections.

2.9 Batch Processing

Definition: Batch Processing

Batch Processing allows multiple SQL statements to be grouped and sent to the database in a *single network round-trip*, dramatically improving performance for bulk INSERT/UPDATE/DELETE operations compared to sending statements individually.

2.9.1 Batch vs. Individual Execution

Performance Comparison

Aspect	Individual Execution	Batch Execution
Network trips	One per statement	One for all
DB overhead	High (parse + execute per row)	Low (bulk parse)
Performance	Slow for large datasets	Up to 10x–100x faster
Use case	Single row operations	Bulk inserts, ETL, data imports

2.9.2 Batch Processing API

Batch Methods

Method	Description
`addBatch(sql)`	Add a SQL string to batch (`Statement`)
`addBatch()`	Add current params to batch (`PreparedStatement`)
`executeBatch()`	Execute all batched statements; returns `int[]`
`clearBatch()`	Remove all commands from the batch

2.9.3 Batch Insert Code Example

Batch Insert with PreparedStatement

```java
String sql = "INSERT INTO students (name, dept, gpa) VALUES (?, ?, ?)";

try (Connection con = DriverManager.getConnection(URL, USER, PASS);
     PreparedStatement ps = con.prepareStatement(sql)) {

    con.setAutoCommit(false);    // Wrap batch in transaction

    String[][] data = {
        {"Alice",   "CS",   "3.8"},
        {"Bob",     "ECE",  "3.5"},
        {"Charlie", "Math", "3.9"},
        {"Diana",   "CS",   "3.7"},
        {"Eve",     "ECE",  "3.6"}
    };

    for (String[] row : data) {
        ps.setString(1, row[0]);
        ps.setString(2, row[1]);
        ps.setDouble(3, Double.parseDouble(row[2]));
        ps.addBatch();                      // Queue these params
    }

    int[] results = ps.executeBatch();   // Send all in one call
    con.commit();

    System.out.println("Inserted: " + results.length + " rows.");

} catch (SQLException e) {
    con.rollback();
    e.printStackTrace();
}
```

2.9.4 Periodic Flush for Very Large Batches

For millions of rows, accumulating the entire batch in memory causes `OutOfMemoryError`. Flush periodically:

Large Batch with Periodic Flush (every 500 rows)

```java
PreparedStatement ps = con.prepareStatement(sql);
con.setAutoCommit(false);
int batchSize = 500;

for (int i = 0; i < 100_000; i++) {
    ps.setString(1, "Student_" + i);
    ps.setString(2, "CS");
    ps.setDouble(3, 3.0 + (i % 10) * 0.1);
    ps.addBatch();

    if ((i + 1) % batchSize == 0) {    // Flush every 500 rows
        ps.executeBatch();
        con.commit();
        ps.clearBatch();
    }
}
ps.executeBatch();   // Flush remaining rows
con.commit();
```

Batch Processing Best Practices

- Always wrap batch in a **transaction** so a failure rolls back the entire batch.
- For very large datasets, **flush periodically** (every 500–1000 rows) to avoid memory issues.
- Check the `int[]` return from `executeBatch()`: a value of `Statement.EXECUTE_FAILED` (–3) signals a failed statement.

2.10 Module 2 Summary

JDBC Complete Reference — Module 2 Summary

Topic	Key Points
Introduction	`java.sql` API; database-independent via driver abstraction
Drivers	Type 4 (Pure Java) is the modern standard; one JAR file
Connection	`DriverManager.getConnection()`; always use try-with-resources
SELECT	`executeQuery()` → `ResultSet`; iterate with `next()`
DML	`executeUpdate()` for INSERT/UPDATE/DELETE; returns rows affected
Statement	Static SQL; simple but SQL-injection vulnerable
PreparedStatement	Parameterised, pre-compiled; safe; use for all user input
CallableStatement	Executes stored procedures; supports IN/OUT parameters
ResultSet	Cursor over rows; access by column name or index
Metadata	`ResultSetMetaData` (column info); `DatabaseMetaData` (DB info)
Transactions	`setAutoCommit(false)` → SQL → `commit()/rollback()`
Savepoints	`setSavepoint()` allows partial rollback within a transaction
Batch Processing	`addBatch() + executeBatch()` for bulk operations; wrap in transaction

Module 2 Key Takeaway

JDBC is the **foundation of all Java database access**. Every higher-level framework — JPA, Hibernate, Spring Data — ultimately generates and executes JDBC calls under the hood. A solid understanding of JDBC is *essential* for debugging, performance tuning, and understanding enterprise Java applications at a fundamental level.

MODULE 3 — Java Server Pages (JSP)

What is JSP?

Java Server Pages (JSP) is a server-side technology that enables developers to create dynamically generated web pages based on HTML, XML, or other document types. JSP pages are *compiled into Servlets* by the web container at runtime, combining the power of Java with the simplicity of HTML templating. JSP is part of the Java EE specification and runs on any compliant web container (e.g. Apache Tomcat, GlassFish).

3.1 Introduction to JSP

JSP was designed to address the shortcoming of embedding large amounts of HTML inside Servlet `PrintWriter` calls, which made presentation logic messy and hard to maintain. JSP inverts this model — HTML is the primary content and Java is *embedded inside* it.

Servlet vs. JSP — Comparison

Aspect	Servlet	JSP
Primary role	Controller / Business logic	View / Presentation layer
Code structure	Java class with HTML in `out.println()`	HTML file with embedded Java
Compilation	Manually compiled	Auto-compiled by container
Ease of design	Harder for HTML design	Easier; HTML-centric
Best suited for	Processing, redirects, filters	Rendering dynamic HTML

JSP in MVC Architecture

In the **MVC** (Model-View-Controller) pattern, JSP serves as the **View**. Servlets act as Controllers, and JavaBeans / EJBs act as the Model. JSP should *not* contain business logic — it should only display data prepared by the Servlet.

3.2 JSP Lifecycle

Every JSP page goes through a well-defined lifecycle managed by the web container. Understanding this lifecycle is fundamental to writing correct JSP code.

JSP Lifecycle Phases

1. **Translation** — The `.jsp` file is translated into a Java Servlet source file (`.java`) by the JSP container.

2. **Compilation** — The generated `.java` file is compiled into a `.class` (bytecode) file.
3. **Loading & Instantiation** — The compiled Servlet class is loaded into the JVM and an instance is created.
4. `jspInit()` — Called *once* when the JSP is first loaded. Used for one-time initialisation (like opening a DB connection).
5. `_jspService()` — Called for *every* HTTP request. Contains the main page logic. Cannot be overridden by the developer.
6. `jspDestroy()` — Called *once* before the JSP is removed from service. Used for cleanup (closing connections, etc.).

JSP Lifecycle Flow

Phase	Method	Called When
Translation	--	`.jsp` is first accessed or modified
Compilation	--	After translation
Initialisation	`jspInit()`	Once, before first request
Request handling	`_jspService()`	Every HTTP request
Destruction	`jspDestroy()`	Before unloading / server shutdown

Overriding jspInit() and jspDestroy()

```
<%!
  // Called once on JSP initialisation
  public void jspInit() {
      System.out.println("JSP Initialised");
  }

  // Called once on JSP destruction
  public void jspDestroy() {
      System.out.println("JSP Destroyed - cleanup done");
  }
%>
```

Key Lifecycle Note

The method `_jspService()` is **auto-generated** by the container and must *not* be defined or overridden in a JSP page. Only `jspInit()` and `jspDestroy()` may be overridden using declaration tags (`<%! ... %>`).

3.3 JSP Syntax and Directives

JSP provides several scripting elements that allow Java code to be embedded in HTML.

JSP Scripting Elements Overview

Tag Syntax	Purpose
`<% ... %>`	Scriptlet — arbitrary Java statements
`<%= ... %>`	Expression — evaluates and prints a value
`<%! ... %>`	Declaration — declares fields and methods
`<%- ... -%->`	JSP comment — not sent to client
`<%@ ... %>`	Directive — page-level configuration

3.4 Scriptlet, Expression and Declaration

3.4.1 Scriptlet <% ... %>

A **scriptlet** contains any valid Java statements. The code inside a scriptlet is placed directly inside the `_jspService()` method of the generated Servlet.

Scriptlet Example

```
<%
  String name = request.getParameter("name");
  int count = 0;
  for (int i = 1; i <= 5; i++) {
      count += i;
  }
  out.println("Sum = " + count);
%>
```

3.4.2 Expression <%= ... %>

A **JSP expression** evaluates a Java expression and inserts the result directly into the output HTML. It must *not* end with a semicolon.

Expression Example

```
<p>Current Time: <%= new java.util.Date() %></p>
<p>Welcome, <%= request.getParameter("username") %>!</p>
<p>Square of 7: <%= 7 * 7 %></p>
```

3.4.3 Declaration <%! ... %>

A **declaration** defines instance variables or methods that become part of the generated Servlet class (outside `_jspService()`). These are shared across all requests.

Declaration Example

```
<%!
  int hitCount = 0;              // Instance variable (shared!)

  String greet(String name) {  // Method declaration
      return "Hello, " + name + "!";
  }
%>

<!-- Use in the page -->
<% hitCount++; %>
<p>Page visits: <%= hitCount %></p>
<p><%= greet("Alice") %></p>
```

Scriptlet vs. Declaration

Variables declared in a **scriptlet** (`<% int x=0; %>`) are *local* to `_jspService()` — a new copy for each request. Variables declared in a **declaration** (`<%! int x=0; %>`) are *instance variables* — shared across all requests and threads. Shared mutable state in declarations requires synchronisation.

3.5 JSP Implicit Objects

JSP automatically makes **nine implicit objects** available inside scriptlets and expressions without any declaration. These are created by the container and passed to `_jspService()`.

The Nine JSP Implicit Objects

Object	Type	Description
request	HttpServletRequest	Current HTTP request; holds parameters, headers, attributes
response	HttpServletResponse	HTTP response; set headers, send redirects
out	JspWriter	Output stream for writing HTML to client
session	HttpSession	Per-user session object for state management
application	ServletContext	Application-wide shared data
config	ServletConfig	Servlet config; init parameters for the JSP
pageContext	PageContext	Access to all scopes; utility methods
page	Object	Reference to the current Servlet instance (this)
exception	Throwable	Available only in error pages; the thrown exception

Using Implicit Objects

```
<!-- request: read form parameter -->
<p>Name: <%= request.getParameter("name") %></p>

<!-- session: store and retrieve values -->
<% session.setAttribute("user", "Alice"); %>
<p>User in session: <%= session.getAttribute("user") %></p>

<!-- application: application-wide counter -->
<%
  Integer visits = (Integer) application.getAttribute("visits");
  if (visits == null) visits = 0;
  application.setAttribute("visits", ++visits);
%>
<p>Total Visits: <%= visits %></p>

<!-- out: write directly to response -->
<% out.println("<b>Hello from out object!</b>"); %>
```

Scope of Implicit Objects

Object	Scope
`page`	Current JSP page only
`request`	Current HTTP request (forwarded pages share it)
`session`	User session (across multiple requests from same browser)
`application`	Entire web application (shared across all users and pages)

3.6 JSP Directives

Directives provide global information about a JSP page to the container. They affect the overall structure of the generated Servlet. Directives use the syntax `<%@ directive attribute="value" %>`.

There are three types of directives:

3.6.1 Page Directive — `<%@ page %>`

The **page directive** defines page-level attributes such as content type, import statements, session management, and error page configuration.

Page Directive Examples

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
         pageEncoding="UTF-8" %>

<%@ page import="java.util.*, java.sql.*" %>

<%@ page session="true" %>

<%@ page errorPage="error.jsp" %>

<%@ page isErrorPage="true" %>

<%@ page isThreadSafe="false" %>
```

Common Page Directive Attributes

Attribute	Description
`language`	Scripting language (default: `java`)
`contentType`	MIME type of the response (e.g. `text/html`)
`import`	Java packages/classes to import
`session`	Whether to participate in a session (`true`/`false`)
`errorPage`	URL of the error handling JSP
`isErrorPage`	Whether this JSP is an error page (`true`/`false`)
`buffer`	Size of the output buffer (e.g. `8kb`)
`isThreadSafe`	Whether the JSP is thread-safe (default: `true`)

3.6.2 Include Directive — `<%@ include %>`

The **include directive** statically includes the content of another file (HTML, JSP, or text) at *translation time*. The included file's content is merged into the current JSP before compilation.

Include Directive

```
<!-- Include a common header at translation time -->
<%@ include file="header.jsp" %>

<h1>Welcome to the Main Page</h1>

<%@ include file="footer.html" %>
```

Include Directive vs. `<jsp:include>` Action

The **include directive** (`<%@ include %>`) is a *static* include at translation time — both files become one Servlet. The **`<jsp:include>`** action is a *dynamic* include at request time — the included page is processed separately and its output is inserted. Use the action when the included content changes per request.

3.6.3 Taglib Directive — `<%@ taglib %>`

The **taglib directive** declares a custom tag library (such as JSTL) for use in the page. It maps a URI to a prefix used to reference tags.

Taglib Directive -- JSTL Example

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>

<c:out value="${message}" />
```

3.7 JSP Action Elements

JSP Action Elements are XML-based tags that control the behaviour of the Servlet engine. Unlike directives (which affect translation), actions execute *at request time*.

Standard JSP Action Elements

Action Tag	Purpose
`<jsp:include>`	Dynamically include another page at request time
`<jsp:forward>`	Forward the request to another resource
`<jsp:param>`	Pass parameters with include/forward
`<jsp:useBean>`	Instantiate or locate a JavaBean
`<jsp:setProperty>`	Set a property on a JavaBean
`<jsp:getProperty>`	Retrieve a property value from a JavaBean
`<jsp:plugin>`	Generate HTML for a Java plugin (applets; deprecated)

3.7.1 `<jsp:include>`

Includes the output of another resource (JSP, Servlet, or HTML) at request time.

jsp:include Example

```
<jsp:include page="navbar.jsp" />

<!-- Include with a parameter -->
<jsp:include page="sidebar.jsp">
    <jsp:param name="section" value="home" />
</jsp:include>
```

3.7.2 `<jsp:forward>`

Forwards the request and response to another resource. Processing stops in the current JSP and transfers to the target.

jsp:forward Example

```
<%
  String role = (String) session.getAttribute("role");
  if (role == null) {
%>
    <jsp:forward page="login.jsp" />
<%
  }
%>
```

3.8 Java Beans in JSP

What is a JavaBean?

A **JavaBean** is a reusable Java class that follows specific conventions:

- Must have a **public no-argument constructor**.
- Properties are accessed via **getter** and **setter** methods following the naming convention `getPropertyName()` / `setPropertyName()`.
- The class should be **serializable** (implements `Serializable`).

JSP provides built-in action tags to work with JavaBeans without scriptlet code.

Example JavaBean -- Student.java

```
package com.example;

public class Student implements java.io.Serializable {
    private String name;
    private int rollNo;
    private double gpa;

    public Student() {}   // Required: no-arg constructor

    public String getName()    { return name; }
    public void setName(String name) { this.name = name; }

    public int getRollNo()     { return rollNo; }
    public void setRollNo(int rollNo) { this.rollNo = rollNo; }

    public double getGpa()     { return gpa; }
    public void setGpa(double gpa) { this.gpa = gpa; }
}
```

3.8.1 Using JavaBeans with JSP Action Tags

jsp:useBean, setProperty, getProperty

```
<!-- Declare / locate the bean -->
<jsp:useBean id="student" class="com.example.Student" scope="session" />

<!-- Set properties explicitly -->
<jsp:setProperty name="student" property="name"   value="Alice" />
<jsp:setProperty name="student" property="rollNo" value="101"   />
<jsp:setProperty name="student" property="gpa"    value="3.9"   />

<!-- Set ALL properties from request parameters automatically -->
<jsp:setProperty name="student" property="*" />

<!-- Get and display a property -->
<p>Name:   <jsp:getProperty name="student" property="name"   /></p>
<p>RollNo: <jsp:getProperty name="student" property="rollNo" /></p>
<p>GPA:    <jsp:getProperty name="student" property="gpa"    /></p>
```

Bean Scope Values in jsp:useBean

Scope	Lifetime
page	Current JSP page only
request	Duration of the current HTTP request
session	User's session (across multiple requests)
application	Entire web application lifetime

3.9 Introduction to JSP Expression Language (EL)

What is Expression Language?

JSP Expression Language (EL), introduced in JSP 2.0, provides a simpler, cleaner syntax to access data stored in JavaBeans, implicit objects, and scoped attributes without using Java scriptlets. EL expressions use the syntax:

```
${expression}
```

3.9.1 EL vs. Scriptlet

Equivalent Code: Scriptlet vs. EL

Scriptlet (old style)	EL (modern style)
`<%= student.getName() %>`	`${student.name}`
`<%= request.getParameter("id") %>`	`${param.id}`
`<%= session.getAttribute("user") %>`	`${sessionScope.user}`

3.9.2 EL Implicit Objects

EL provides its own set of implicit objects for accessing scoped data:

EL Implicit Objects

EL Object	Description
`param`	Request parameters (single value)
`paramValues`	Request parameters (array of values)
`header`	HTTP request headers
`cookie`	Cookie values
`pageScope`	Attributes in page scope
`requestScope`	Attributes in request scope
`sessionScope`	Attributes in session scope
`applicationScope`	Attributes in application scope
`initParam`	Context init parameters

3.9.3 EL Operators

EL Operators

Category	Operators	Example
Arithmetic	`+ - * / %`	`${5 + 3}` → 8
Relational	`== != < > <= >=`	`${age >= 18}`
Logical	`&& || !`	`${a && b}`
Conditional	`? :`	`${x > 0 ? "pos" : "neg"}`
Empty	`empty`	`${empty list}`

EL Usage Examples

```
<!-- Accessing a bean property -->
<p>Student Name: ${student.name}</p>

<!-- Accessing a map / request attribute -->
<p>User: ${sessionScope.loggedInUser}</p>

<!-- Request parameter -->
<p>Search: ${param.query}</p>

<!-- Arithmetic -->
<p>Price with Tax: ${product.price * 1.18}</p>

<!-- Conditional -->
<p>${age >= 18 ? "Adult" : "Minor"}</p>

<!-- Empty check -->
<c:if test="${empty cart.items}">
    <p>Your cart is empty.</p>
</c:if>
```

3.10 Introduction to JSTL Core Tags

What is JSTL?

JSTL (JavaServer Pages Standard Tag Library) is a collection of useful JSP tags that encapsulate core functionality — iteration, conditionals, URL handling, formatting, and more — to *eliminate scriptlet code* from JSP pages entirely. To use JSTL, add the taglib directive:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

3.10.1 `<c:out>` — Output a Value

Safely outputs a value to the page, escaping XML/HTML special characters by default.

c:out

```
<c:out value="${user.name}" />

<!-- With default if null -->
<c:out value="${product.description}" default="No description available." />

<!-- Disable escaping (output raw HTML) -->
<c:out value="${htmlContent}" escapeXml="false" />
```

3.10.2 <c:set> — Set a Variable

Sets the value of an EL variable or a scoped attribute.

c:set

```
<!-- Set a local EL variable -->
<c:set var="discount" value="0.15" />
<p>Discount: ${discount * 100}%</p>

<!-- Set a scoped attribute -->
<c:set var="username" value="Alice" scope="session" />

<!-- Set a bean property -->
<c:set target="${student}" property="name" value="Bob" />
```

3.10.3 <c:remove> — Remove a Variable

Removes a scoped attribute.

c:remove

```
<c:remove var="tempData" scope="request" />
<c:remove var="username" scope="session" />
```

3.10.4 <c:if> — Conditional Rendering

Evaluates a condition and renders its body only if the condition is `true`. Unlike Java's `if-else`, there is no `<c:else>` — use `<c:choose>` for that.

c:if

```
<c:if test="${not empty sessionScope.user}">
    <p>Welcome, ${sessionScope.user}!</p>
</c:if>

<c:if test="${product.stock == 0}">
    <span style="color:red;">Out of Stock</span>
</c:if>
```

3.10.5 <c:choose>, <c:when>, <c:otherwise> — if-else Chain

Provides a multi-branch conditional, equivalent to Java's `if-else if-else`.

c:choose / c:when / c:otherwise

```
<c:choose>
    <c:when test="${score >= 90}">
        <p>Grade: A</p>
```

```
    </c:when>
    <c:when test="${score >= 75}">
        <p>Grade: B</p>
    </c:when>
    <c:when test="${score >= 60}">
        <p>Grade: C</p>
    </c:when>
    <c:otherwise>
        <p>Grade: F (Fail)</p>
    </c:otherwise>
</c:choose>
```

3.10.6 <c:forEach> — Iteration

Iterates over a collection (List, Set, array, Map) or a numeric range.

c:forEach -- Collection and Numeric Range

```
<!-- Iterate over a List of beans -->
<table>
  <tr><th>Name</th><th>GPA</th></tr>
  <c:forEach var="s" items="${studentList}">
    <tr>
      <td>${s.name}</td>
      <td>${s.gpa}</td>
    </tr>
  </c:forEach>
</table>

<!-- Numeric range (begin, end, step) -->
<c:forEach var="i" begin="1" end="5" step="1">
    <p>Item ${i}</p>
</c:forEach>

<!-- Access loop status -->
<c:forEach var="item" items="${items}" varStatus="status">
    <p>${status.index + 1}. ${item}
       ${status.last ? "(last)" : ""}</p>
</c:forEach>
```

3.10.7 <c:forTokens> — Tokenize a String

Iterates over tokens in a string, splitting by a specified delimiter.

c:forTokens

```
<!-- Split a comma-separated string -->
<c:forTokens var="fruit" items="apple,mango,banana,grape" delims=",">
    <li>${fruit}</li>
</c:forTokens>

<!-- Multiple delimiters -->
<c:forTokens var="word" items="Java:EE;JSP-JSTL" delims=":;-">
    <span>${word} </span>
</c:forTokens>
```

3.10.8 <c:import> — Import a Resource

Imports the content of a URL or resource (local or remote) into the current page at request time. More powerful than `<jsp:include>` as it can import external URLs.

c:import

```
<!-- Import a local JSP fragment -->
<c:import url="/WEB-INF/fragments/header.jsp" />

<!-- Import an external resource -->
<c:import url="https://example.com/terms.html" />

<!-- Store in a variable instead of printing -->
<c:import url="/data/config.xml" var="xmlData" />
```

3.10.9 <c:url> — URL Encoding

Constructs a properly encoded URL, automatically adding session ID if cookies are disabled.

c:url

```
<!-- Simple URL -->
<a href="<c:url value='/product/list' />">View Products</a>

<!-- URL with parameters -->
<c:url value="/product/detail" var="detailUrl">
    <c:param name="id"       value="${product.id}" />
    <c:param name="category" value="electronics"   />
</c:url>
<a href="${detailUrl}">Product Details</a>
```

3.10.10 <c:redirect> — Redirect the Browser

Sends an HTTP redirect (302) response to the client, forwarding to a new URL.

c:redirect

```
<!-- Redirect if not logged in -->
<c:if test="${empty sessionScope.user}">
    <c:redirect url="/login.jsp" />
</c:if>

<!-- Redirect with parameters -->
<c:redirect url="/dashboard.jsp">
    <c:param name="msg" value="LoginSuccess" />
</c:redirect>
```

3.11 JSTL Functions

JSTL provides a set of standard **EL functions** for common string operations. Import the functions library with:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
```

JSTL fn: Functions Reference

Function	Description
`fn:length(obj)`	Returns length of string or collection
`fn:toUpperCase(str)`	Converts string to upper case
`fn:toLowerCase(str)`	Converts string to lower case
`fn:trim(str)`	Removes leading/trailing whitespace
`fn:substring(str,start,end)`	Extracts substring (end exclusive)
`fn:substringBefore(str,sub)`	Substring before first occurrence of `sub`
`fn:substringAfter(str,sub)`	Substring after first occurrence of `sub`
`fn:replace(str,old,new)`	Replaces all occurrences of `old` with `new`
`fn:split(str,delim)`	Splits string into an array by delimiter
`fn:join(array,sep)`	Joins array elements with separator
`fn:contains(str,sub)`	Returns `true` if `str` contains `sub`
`fn:containsIgnoreCase(s,sub)`	Case-insensitive contains check
`fn:startsWith(str,prefix)`	Returns `true` if `str` starts with `prefix`
`fn:endsWith(str,suffix)`	Returns `true` if `str` ends with `suffix`
`fn:indexOf(str,sub)`	Index of first occurrence of `sub`
`fn:escapeXml(str)`	Escapes XML special characters

JSTL fn: Usage Examples

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>

<p>Length: ${fn:length(studentList)}</p>

<p>Upper: ${fn:toUpperCase("hello world")}</p>

<p>Contains 'Java': ${fn:contains(course, "Java")}</p>

<p>Starts with 'Web': ${fn:startsWith(title, "Web")}</p>

<p>Substring: ${fn:substring("JavaEEProgramming", 0, 6)}</p>

<!-- Splitting a string into an array and iterating -->
```

```
<c:set var="tags" value="${fn:split(tagString, ',')}" />
<c:forEach var="tag" items="${tags}">
    <span class="tag">${fn:trim(tag)}</span>
</c:forEach>
```

3.12 Custom Tag Library

What is a Custom Tag Library?

A **Custom Tag Library** allows developers to create their own JSP tags, encapsulating complex Java logic behind a simple XML-like tag. Custom tags extend JSP beyond JSTL and help in writing clean, reusable, and maintainable view code.

3.12.1 Components of a Custom Tag

Custom Tag Components

Component	Description
Tag Handler Class	Java class that implements the tag's logic (extends `SimpleTagSupport` or `TagSupport`)
TLD File	Tag Library Descriptor (`.tld`) — XML file mapping tag names to handler classes
Taglib Directive	Used in JSP to declare and use the custom library

3.12.2 Step 1 — Create the Tag Handler Class

Tag handler classes extend `SimpleTagSupport` (JSP 2.0+, recommended).

HelloTag.java -- Simple Custom Tag

```java
package com.example.tags;

import javax.servlet.jsp.tagext.SimpleTagSupport;
import javax.servlet.jsp.JspWriter;
import java.io.IOException;

public class HelloTag extends SimpleTagSupport {

    private String name;  // Custom attribute

    public void setName(String name) { this.name = name; }

    @Override
```

```
    public void doTag() throws javax.servlet.jsp.JspException, IOException {
        JspWriter out = getJspContext().getOut();
        out.print("<h2>Hello, " + name + "! Welcome to Custom Tags.</h2>");
    }
}
```

3.12.3 Step 2 — Create the TLD File

The **Tag Library Descriptor** (`.tld`) is an XML file placed in the `WEB-INF/` directory that registers the tag handler.

mytags.tld -- Tag Library Descriptor

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib xmlns="http://java.sun.com/xml/ns/javaee" version="2.1">

    <tlib-version>1.0</tlib-version>
    <short-name>mytags</short-name>
    <uri>http://myapp.com/tags</uri>

    <tag>
        <name>hello</name>
        <tag-class>com.example.tags.HelloTag</tag-class>
        <body-content>empty</body-content>

        <attribute>
            <name>name</name>
            <required>true</required>
            <rtexprvalue>true</rtexprvalue>  <!-- Allow EL expressions -->
        </attribute>
    </tag>

</taglib>
```

3.12.4 Step 3 — Use the Custom Tag in a JSP

Using the Custom Tag in JSP

```
<%@ taglib uri="http://myapp.com/tags" prefix="my" %>

<!DOCTYPE html>
<html>
<body>
    <!-- Using our custom hello tag -->
    <my:hello name="Alice" />
    <my:hello name="${sessionScope.username}" />
</body>
```

```
</html>
```

3.12.5 Custom Tag with Body Content

Tags can also process body content (text between opening and closing tag).

BoxTag.java -- Tag with Body

```java
package com.example.tags;

import javax.servlet.jsp.tagext.SimpleTagSupport;
import java.io.*;

public class BoxTag extends SimpleTagSupport {
    private String color = "#003577";   // Default: Yale Blue

    public void setColor(String color) { this.color = color; }

    @Override
    public void doTag() throws javax.servlet.jsp.JspException, IOException {
        JspWriter out = getJspContext().getOut();
        out.print("<div style='border:2px solid " + color +
                  "; padding:10px; border-radius:6px;'>");
        getJspBody().invoke(null);   // Render the body content
        out.print("</div>");
    }
}
```

Using BoxTag in JSP

```jsp
<my:box color="#A3873C">
    <h3>This content is inside a custom styled box!</h3>
    <p>Custom tags can wrap and style any body content.</p>
</my:box>
```

body-content Values in TLD

- `empty` — Tag must have no body.
- `scriptless` — Body may contain EL and standard actions, no scriptlets.
- `tagdependent` — Body is treated as literal text (not evaluated).
- `JSP` — Body can contain any JSP content (deprecated for `SimpleTagSupport`).

3.13 Module 3 Summary

JSP Complete Reference — Module 3 Summary

Topic	Key Points
Introduction	JSP = HTML + embedded Java; compiled to Servlet; used as View in MVC
Lifecycle	Translation → Compilation → Init (`jspInit`) → Service (`_jspService`) → Destroy (`jspDestroy`)
Scriptlet	`<% %>` — Java statements placed in `_jspService()`
Expression	`<%= %>` — Evaluates and prints a value; no semicolon
Declaration	`<%! %>` — Instance variables/methods; shared across requests
Implicit Objects	9 auto-provided objects: `request`, `response`, `out`, `session`, `application`, `config`, `pageContext`, `page`, `exception`
Directives	`page` (config), `include` (static merge), `taglib` (tag library)
Action Elements	`jsp:include` (dynamic), `jsp:forward`, `jsp:useBean`, `jsp:setProperty`, `jsp:getProperty`
JavaBeans	POJO with getters/setters; used with `jsp:useBean` for clean MVC
EL	`${expr}` — Replaces scriptlets; accesses beans, scopes, params
JSTL Core	`c:out`, `c:set`, `c:remove`, `c:if`, `c:choose`, `c:forEach`, `c:forTokens`, `c:import`, `c:url`, `c:redirect`
JSTL Functions	`fn:` prefix; string utilities (length, split, contains, upper, etc.)
Custom Tags	Tag Handler + TLD file + taglib directive; encapsulate reusable logic

Module 3 Key Takeaway

JSP, EL, and JSTL together form the **standard View layer** of Java EE web applications. The modern best practice is: *zero scriptlets in JSP* — use EL for data access and JSTL for control flow. Custom tag libraries further promote reusability and separation of concerns, keeping presentation clean and Java logic confined to Servlets and JavaBeans.

MODULE 4 — Servlets

What is a Servlet?

A **Servlet** is a Java class that runs on a web/application server and handles client requests (typically HTTP) to produce dynamic responses. Servlets extend `HttpServlet` and are managed by a **Servlet Container** (e.g. Apache Tomcat). They form the **Controller** layer in Java EE MVC architecture.

4.1 Introduction to Servlets

CGI vs. Servlet

Aspect	CGI	Servlet
Process model	New process per request	Single JVM instance; threads per request
Performance	Slow (process creation overhead)	Fast (thread-based)
State sharing	Not possible across requests	Possible via `ServletContext`
Platform	Platform dependent	Platform independent (Java)

A Servlet must import `javax.servlet.*` and `javax.servlet.http.*`. Every HTTP Servlet extends `HttpServlet` and overrides `doGet()`, `doPost()`, or both.

Minimal HttpServlet

```java
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.*;
import java.io.*;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h1>Hello from Servlet!</h1>");
    }
}
```

4.2 Servlet Life Cycle

Servlet Lifecycle — Three Phases

1. **Loading & Instantiation** — Container loads the class and creates one instance.
2. `init(ServletConfig)` — Called *once*; one-time setup (DB connections, config loading).
3. `service(req, res)` — Called for *every request*; dispatches to `doGet()`, `doPost()`, etc.
4. `destroy()` — Called *once* before unloading; cleanup.

Lifecycle Methods

Method	Called When
`init()`	Once, when Servlet is first loaded
`service()`	Every HTTP request; routes to doGet/doPost
`doGet()`	HTTP GET request
`doPost()`	HTTP POST request
`destroy()`	Once, before Servlet is removed

Servlet Lifecycle Override

```java
@WebServlet("/lifecycle")
public class LifecycleServlet extends HttpServlet {

    @Override
    public void init() throws ServletException {
        System.out.println("Servlet initialised once.");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.getWriter().println("Handling GET request.");
    }

    @Override
    public void destroy() {
        System.out.println("Servlet destroyed.");
    }
}
```

4.3 Handling HTTP Requests and Responses

4.3.1 HttpServletRequest

Key HttpServletRequest Methods

Method	Description
`getParameter(name)`	Single form/query parameter value
`getParameterValues(name)`	Array of values (e.g. checkboxes)
`getParameterMap()`	All parameters as a `Map`
`getAttribute(name)`	Request-scoped attribute
`setAttribute(name, val)`	Store request-scoped attribute
`getSession()`	Get/create `HttpSession`
`getHeader(name)`	HTTP header value
`getMethod()`	Returns `"GET"`, `"POST"`, etc.
`getRequestDispatcher(url)`	Get dispatcher for forward/include

4.3.2 HttpServletResponse

Key HttpServletResponse Methods

Method	Description
`setContentType(type)`	Set MIME type (e.g. `text/html`)
`getWriter()`	Returns `PrintWriter` for text output
`getOutputStream()`	Returns `ServletOutputStream` for binary output
`sendRedirect(url)`	Send HTTP 302 redirect to client
`setHeader(name, val)`	Set response header
`setStatus(code)`	Set HTTP status code
`sendError(code, msg)`	Send error response

Handling GET and POST with Form Data

```java
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        String user = req.getParameter("username");
        String pass = req.getParameter("password");

        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

```

```
        if ("admin".equals(user) && "1234".equals(pass)) {
            req.getSession().setAttribute("user", user);
            res.sendRedirect("dashboard.jsp");
        } else {
            out.println("<p>Invalid credentials.</p>");
        }
    }
}
```

4.3.3 RequestDispatcher — Forward vs. Redirect

Forward vs. Redirect

Aspect	forward()	sendRedirect()
Server/Client side	Server-side; transparent to client	Client-side; browser gets new URL
URL change	No URL change in browser	URL changes in browser
Request attributes	Shared (same request object)	Lost (new request)
Speed	Faster	Slightly slower (extra round-trip)
Use case	Pass to JSP for rendering	Redirect after POST (PRG pattern)

Forward to JSP

```
// Forward: share request attributes with JSP
req.setAttribute("students", studentList);
RequestDispatcher rd = req.getRequestDispatcher("students.jsp");
rd.forward(req, res);
```

4.4 Servlet Deployment Descriptor (`web.xml`)

Purpose of `web.xml`

The **Deployment Descriptor** (`web.xml`), located in `WEB-INF/`, is an XML configuration file for the web application. It declares Servlets, maps URLs, sets init parameters, configures welcome pages, error pages, filters, and security constraints. Since Servlet 3.0, `@WebServlet` annotations can replace most `web.xml` configurations.

web.xml -- Servlet Declaration and Mapping

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">

    <!-- Servlet Declaration -->
    <servlet>
        <servlet-name>HelloServlet</servlet-name>
        <servlet-class>com.example.HelloServlet</servlet-class>
        <init-param>
            <param-name>adminEmail</param-name>
            <param-value>admin@example.com</param-value>
        </init-param>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <!-- URL Mapping -->
    <servlet-mapping>
        <servlet-name>HelloServlet</servlet-name>
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>

    <!-- Welcome File -->
    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>

    <!-- Error Page -->
    <error-page>
        <error-code>404</error-code>
        <location>/error404.jsp</location>
    </error-page>

    <!-- Context Init Parameter (ServletContext) -->
    <context-param>
        <param-name>dbUrl</param-name>
        <param-value>jdbc:mysql://localhost/mydb</param-value>
    </context-param>

</web-app>
```

Annotation vs. web.xml

`@WebServlet("/path")` on the Servlet class is equivalent to the `<servlet>` + `<servlet-mapping>` block in `web.xml`. Use `web.xml` when you need centralised configuration, overrides, or compatibility with older containers.

4.5 ServletContext and ServletConfig

4.5.1 ServletConfig

`ServletConfig` provides configuration specific to *one* Servlet, available only during `init()`. It holds init parameters defined in `web.xml` or via `@WebInitParam`.

ServletConfig Example

```java
@WebServlet(value="/config",
    initParams = @WebInitParam(name="timeout", value="30"))
public class ConfigServlet extends HttpServlet {

    private String timeout;

    @Override
    public void init(ServletConfig config) throws ServletException {
        super.init(config);
        timeout = config.getInitParameter("timeout");
        System.out.println("Timeout: " + timeout);
    }
}
```

4.5.2 ServletContext

`ServletContext` is shared across the *entire web application* — all Servlets and JSPs can read and write to it. Obtained via `getServletContext()`.

ServletContext Example

```java
// Reading context init param (from web.xml <context-param>)
String dbUrl = getServletContext().getInitParameter("dbUrl");

// Store application-wide data
getServletContext().setAttribute("visitCount", 0);

// Read it in another Servlet or JSP
Integer count =
    (Integer) getServletContext().getAttribute("visitCount");
```

ServletConfig vs. ServletContext

Aspect	ServletConfig	ServletContext
Scope	One Servlet only	Entire web application
Params from	`<init-param>`	`<context-param>`
Created by	Container per Servlet	Container once per application
Access via	`getServletConfig()`	`getServletContext()`

4.6 Session Management and Cookies

HTTP is stateless. **Session management** techniques allow state to be maintained across multiple requests from the same client.

Session Management Techniques

Technique	Description
Cookies	Small key-value text stored in browser; sent with every request
HttpSession	Server-side storage; session ID sent via cookie or URL rewriting
URL Rewriting	Session ID appended to URL (fallback if cookies disabled)
Hidden Fields	Session data embedded in HTML form fields

4.6.1 Cookies

Creating and Reading Cookies

```java
// Setting a cookie
Cookie c = new Cookie("username", "Alice");
c.setMaxAge(60 * 60 * 24);   // 1 day in seconds
c.setPath("/");
res.addCookie(c);

// Reading cookies
Cookie[] cookies = req.getCookies();
if (cookies != null) {
    for (Cookie ck : cookies) {
        if ("username".equals(ck.getName())) {
            out.println("Welcome back, " + ck.getValue());
        }
    }
```

```
}

// Deleting a cookie (set maxAge to 0)
Cookie del = new Cookie("username", "");
del.setMaxAge(0);
res.addCookie(del);
```

4.6.2 HttpSession

`HttpSession` is the preferred server-side mechanism. The container manages a session ID (usually via a `JSESSIONID` cookie).

HttpSession Usage

```
// Create or retrieve session
HttpSession session = req.getSession();          // creates if absent
HttpSession session = req.getSession(false);     // returns null if absent

// Store and retrieve attributes
session.setAttribute("cart", cartObject);
ShoppingCart cart = (ShoppingCart) session.getAttribute("cart");

// Session info
String id  = session.getId();
long created = session.getCreationTime();
session.setMaxInactiveInterval(30 * 60);  // 30-minute timeout

// Invalidate (logout)
session.invalidate();
```

Cookie vs. HttpSession

Cookies store data client-side (size limit ≈4 KB; visible in browser). **HttpSession** stores data server-side (no size limit; secure). For sensitive data (user IDs, cart contents), always use `HttpSession`.

4.7 Servlet Chaining and Filters

4.7.1 Servlet Chaining

Servlet Chaining is the technique of routing a request through a sequence of Servlets where the output of one becomes the input of the next, using `RequestDispatcher`.

Chaining with RequestDispatcher

```java
// ServletA processes, then forwards to ServletB
req.setAttribute("processedData", result);
RequestDispatcher rd = req.getRequestDispatcher("/servletB");
rd.forward(req, res);
```

4.7.2 Servlet Filters

A **Filter** is a reusable component that intercepts requests and responses *before* they reach the Servlet (or after). Filters implement `javax.servlet.Filter`.

Common Filter Use Cases

- Authentication / Authorisation checks
- Logging and auditing requests
- Request/Response encoding (e.g. force UTF-8)
- Data compression (GZIP)
- Cross-Origin Resource Sharing (CORS) headers

Filter Implementation

```java
import javax.servlet.*;
import javax.servlet.annotation.WebFilter;
import java.io.*;

@WebFilter("/*")          // Intercept all requests
public class AuthFilter implements Filter {

    @Override
    public void init(FilterConfig fc) throws ServletException {}

    @Override
    public void doFilter(ServletRequest req, ServletResponse res,
                         FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest request  = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;

        HttpSession session = request.getSession(false);
        String uri = request.getRequestURI();

        boolean isLoginPage = uri.endsWith("login.jsp")
                            || uri.endsWith("LoginServlet");

        if (!isLoginPage && (session == null
```

```
                || session.getAttribute("user") == null)) {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        } else {
            chain.doFilter(req, res);   // Pass to next filter/Servlet
        }
    }

    @Override
    public void destroy() {}
}
```

Filter Lifecycle

Method	Description
`init(FilterConfig)`	Called once; read filter init params
`doFilter(req, res, chain)`	Called per request; call `chain.doFilter()` to continue
`destroy()`	Called once on shutdown

Filter Chain

Multiple filters can be chained. The container executes them in the order they are declared in `web.xml` (or by the `@WebFilter` URL pattern scope). Each filter *must* call `chain.doFilter()` to pass control to the next filter; omitting this call blocks the request.

4.8 File Upload and Download in Servlet

4.8.1 File Upload

Since Servlet 3.0, file upload is natively supported via the `@MultipartConfig` annotation and the `Part` API — no external library required.

File Upload Servlet

```
import javax.servlet.annotation.*;
import javax.servlet.http.*;
import java.io.*;

@WebServlet("/upload")
@MultipartConfig(
    fileSizeThreshold = 1024 * 1024,      // 1 MB -- use temp file above this
    maxFileSize       = 10 * 1024 * 1024, // 10 MB max per file
    maxRequestSize    = 50 * 1024 * 1024  // 50 MB max total request
)
```

```java
public class FileUploadServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        req.setCharacterEncoding("UTF-8");

        // Read a regular form field
        String description = req.getParameter("description");

        // Read the uploaded file
        Part filePart = req.getPart("file");
        String fileName = filePart.getSubmittedFileName();

        // Save to server disk
        String savePath = getServletContext().getRealPath("") + "/uploads/";
        new File(savePath).mkdirs();
        filePart.write(savePath + fileName);

        res.getWriter().println("Uploaded: " + fileName);
    }
}
```

HTML Form for File Upload

```html
<form action="upload" method="post" enctype="multipart/form-data">
    <input type="text"  name="description" placeholder="Description" />
    <input type="file"  name="file" />
    <button type="submit">Upload</button>
</form>
```

enctype Attribute

The form **must** use `enctype="multipart/form-data"` for file uploads. Without this, only the filename string is sent, not the file content.

4.8.2 File Download

For file downloads, set the response content type to the file's MIME type and the `Content-Disposition` header to `attachment`.

File Download Servlet

```java
@WebServlet("/download")
public class FileDownloadServlet extends HttpServlet {

```

```java
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        String fileName = req.getParameter("file");
        String filePath = getServletContext().getRealPath("/uploads/")
                          + fileName;

        File file = new File(filePath);
        if (!file.exists()) {
            res.sendError(HttpServletResponse.SC_NOT_FOUND);
            return;
        }

        // Set headers
        res.setContentType(getServletContext().getMimeType(fileName));
        res.setContentLengthLong(file.length());
        res.setHeader("Content-Disposition",
                      "attachment; filename=\"" + fileName + "\"");

        // Stream file bytes to client
        try (FileInputStream fis = new FileInputStream(file);
             OutputStream out   = res.getOutputStream()) {
            byte[] buf = new byte[4096];
            int   read;
            while ((read = fis.read(buf)) != -1) {
                out.write(buf, 0, read);
            }
        }
    }
}
```

Key Download Response Headers

Header	Purpose
`Content-Type`	MIME type of the file (e.g. `application/pdf`)
`Content-Disposition`	`attachment` triggers browser download prompt
`Content-Length`	File size in bytes; allows browser to show progress

4.9 Module 4 Summary

Servlet Complete Reference — Module 4 Summary

Topic	Key Points
Introduction	Extends `HttpServlet`; thread-based; Controller in MVC
Lifecycle	`init()` → `service()` → `doGet/doPost()` → `destroy()`
Request/Response	`getParameter()`, `setAttribute()`, `sendRedirect()`, `forward()`
web.xml	Servlet mapping, init params, welcome/error pages, context params
ServletConfig	Per-Servlet init parameters; available in `init()`
ServletContext	Application-wide shared data and context params
Session / Cookie	`HttpSession` for server-side state; Cookie for client-side
Filters	Pre/post-process requests; auth, logging, encoding
Chaining	RequestDispatcher `forward()` to pass between Servlets
File Upload	`@MultipartConfig` + `Part` API; form needs `multipart/form-data`
File Download	Set `Content-Disposition: attachment`; stream bytes via `getOutputStream()`

Module 4 Key Takeaway

Servlets are the **backbone of Java EE web development**. Every JSP compiles into a Servlet; every framework (Spring MVC, Struts) sits on top of the Servlet API. Mastering the request-response cycle, session management, filters, and the deployment descriptor gives you complete control over the HTTP layer in any Java web application.

MODULE 5 — JSP and Servlet — Integration

Why Combine JSP and Servlets?

Neither JSP nor Servlet alone is ideal for a complete web application. **Servlets** are powerful for processing logic but produce messy HTML via `PrintWriter`. **JSP** is clean for presentation but becomes unmaintainable when filled with scriptlets. Combining them — Servlet as Controller, JSP as View — gives a clean, maintainable, and scalable architecture.

5.1 Combining JSP and Servlets

The standard pattern is:

1. Client sends a request to a **Servlet**.
2. Servlet processes the request, executes business logic, prepares data.
3. Servlet places results as **request attributes**.
4. Servlet **forwards** to a JSP.
5. JSP reads the attributes and renders the HTML response.

Responsibility Division

Concern	Handled By	Why
URL routing, validation	Servlet	Java is better for logic
Business logic, DB calls	Servlet / Model (Bean)	No HTML mixed with logic
HTML rendering, display	JSP + EL + JSTL	Clean template; no Java

Step 1 -- HTML Form (client)

```html
<!-- index.html -->
<form action="StudentServlet" method="post">
    <input type="text" name="name"   placeholder="Name" />
    <input type="text" name="rollNo" placeholder="Roll No" />
    <button type="submit">Submit</button>
</form>
```

Step 2 -- Servlet (Controller): process and forward

```java
@WebServlet("/StudentServlet")
public class StudentServlet extends HttpServlet {

    @Override
```

```java
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        // 1. Read input
        String name   = req.getParameter("name");
        int    rollNo = Integer.parseInt(req.getParameter("rollNo"));

        // 2. Business logic / model interaction
        Student s = new Student(name, rollNo);   // JavaBean
        // e.g. studentDAO.save(s);

        // 3. Place result in request scope
        req.setAttribute("student", s);

        // 4. Forward to JSP (View)
        RequestDispatcher rd =
            req.getRequestDispatcher("result.jsp");
        rd.forward(req, res);
    }
}
```

Step 3 -- JSP (View): display data with EL + JSTL

```jsp
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<body>
    <h2>Student Registered</h2>
    <p>Name:    <c:out value="${student.name}"   /></p>
    <p>Roll No: <c:out value="${student.rollNo}" /></p>
</body>
</html>
```

Zero Scriptlets in JSP

In a properly combined JSP-Servlet application, the JSP should contain **no scriptlets** (`<% %>`). All Java logic belongs in the Servlet. The JSP uses only EL (`${...}`) and JSTL tags for display.

5.2 Model-View-Controller (MVC) Architecture

MVC in Java EE

MVC separates an application into three interconnected components so that each evolves independently. In Java EE web applications:

- **Model** — JavaBeans / POJOs / DAO classes holding data and business logic.
- **View** — JSP pages using EL and JSTL for rendering HTML.
- **Controller** — Servlet that receives requests, invokes the model, and selects the view.

MVC Component Responsibilities

Role	Technology	Responsibilities
Model	`JavaBean / DAO`	Hold data, DB access, business rules, validation
View	`JSP + EL + JSTL`	Render HTML; read model data; no Java logic
Controller	`HttpServlet`	Parse request, call model, set attributes, choose view

5.2.1 MVC Request Flow

1. **Client** sends HTTP request to a URL mapped to a Servlet.
2. **Controller (Servlet)** reads parameters, validates input.
3. Controller calls **Model** (DAO / service) to fetch or persist data.
4. Model returns a result object (JavaBean, List, etc.).
5. Controller stores result in request/session scope via `setAttribute()`.
6. Controller forwards to **View (JSP)** using `RequestDispatcher`.
7. View renders the HTML using EL and JSTL; response sent to client.

5.2.2 Complete MVC Example — Student List

Model -- Student.java (JavaBean)

```java
package com.example.model;

public class Student {
    private int    rollNo;
    private String name;
    private double gpa;

    public Student(int rollNo, String name, double gpa) {
        this.rollNo = rollNo;
        this.name   = name;
        this.gpa    = gpa;
    }
    // Getters and setters omitted for brevity
```

```
    public int    getRollNo() { return rollNo; }
    public String getName()   { return name;   }
    public double getGpa()    { return gpa;    }
}
```

Model -- StudentDAO.java (Data Access)

```
package com.example.model;

import java.util.*;

public class StudentDAO {
    // Simulated data source (replace with real JDBC calls)
    public List<Student> getAllStudents() {
        List<Student> list = new ArrayList<>();
        list.add(new Student(101, "Alice", 3.9));
        list.add(new Student(102, "Bob",   3.5));
        list.add(new Student(103, "Carol", 3.7));
        return list;
    }
}
```

Controller -- StudentController.java (Servlet)

```
package com.example.controller;

import com.example.model.*;
import javax.servlet.*;
import javax.servlet.annotation.*;
import javax.servlet.http.*;
import java.io.*;
import java.util.List;

@WebServlet("/students")
public class StudentController extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        // Call Model
        StudentDAO dao  = new StudentDAO();
        List<Student> students = dao.getAllStudents();

        // Pass data to View
        req.setAttribute("students", students);
```

```java

        // Forward to View
        req.getRequestDispatcher("/WEB-INF/views/studentList.jsp")
            .forward(req, res);
    }
}
```

View -- studentList.jsp (JSP)

```jsp
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head><title>Student List</title></head>
<body>
<h2>All Students</h2>
<table border="1">
    <tr><th>Roll No</th><th>Name</th><th>GPA</th></tr>
    <c:forEach var="s" items="${students}">
        <tr>
            <td>${s.rollNo}</td>
            <td>${s.name}</td>
            <td>${s.gpa}</td>
        </tr>
    </c:forEach>
</table>
</body>
</html>
```

Place JSPs in WEB-INF/views/

Storing JSPs inside `WEB-INF/` makes them **inaccessible directly via URL** — they can only be accessed through a Servlet forward. This enforces the MVC rule that all requests go through the Controller first.

5.2.3 Front Controller Pattern

In larger applications a single **Front Controller Servlet** handles all requests and delegates to specific action classes — the pattern used by frameworks like Spring MVC (`DispatcherServlet`) and Struts.

Front Controller -- FrontController.java

```java
@WebServlet("/app/*")
public class FrontController extends HttpServlet {

    @Override
```

```java
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        route(req, res);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        route(req, res);
    }

    private void route(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        String path = req.getPathInfo();   // e.g. /students, /login

        switch (path) {
            case "/students":
                new StudentController().doGet(req, res); break;
            case "/login":
                new LoginController().doPost(req, res);  break;
            default:
                res.sendError(HttpServletResponse.SC_NOT_FOUND);
        }
    }
}
```

5.3 Forwarding Requests Between JSP and Servlet

There are two server-side mechanisms to transfer a request to another resource: **forward** and **include**, both via `RequestDispatcher`.

Forward vs. Include vs. Redirect

Aspect	`forward()`	`include()`	`sendRedirect()`
Side	Server-side	Server-side	Client-side (new request)
URL change	No	No	Yes
Control	Transferred entirely	Returns after include	New request
Request attributes	Shared	Shared	Lost
Use case	Route to View	Reusable fragments	Post-redirect-get

5.3.1 `RequestDispatcher.forward()` — Servlet to JSP

Servlet forwarding to JSP

```
// In Servlet doGet/doPost:
req.setAttribute("message", "Data loaded successfully.");
req.setAttribute("items",   itemList);

RequestDispatcher rd = req.getRequestDispatcher("/WEB-INF/views/result.jsp");
rd.forward(req, res);
// Execution of the Servlet stops here; JSP takes over.
```

5.3.2 `RequestDispatcher.include()` — Embedding Fragments

Including a JSP fragment inside another Servlet/JSP

```
// In Servlet --- include a JSP fragment into the current response
RequestDispatcher rd = req.getRequestDispatcher("/header.jsp");
rd.include(req, res);

// In JSP --- include another JSP's output inline
// (equivalent to <jsp:include>)
<% request.getRequestDispatcher("/sidebar.jsp").include(request, response);
   %>
```

5.3.3 JSP forwarding back to a Servlet

JSP can also forward to a Servlet when user interaction requires re-processing.

JSP forwarding to a Servlet

```
<!-- Using jsp:forward action (preferred) -->
<c:if test="${empty sessionScope.user}">
    <jsp:forward page="/LoginServlet" />
</c:if>

<!-- Or via scriptlet (avoid in practice) -->
<%
  if (session.getAttribute("user") == null) {
      request.getRequestDispatcher("/LoginServlet")
              .forward(request, response);
  }
%>
```

5.3.4 Passing Data with Request Attributes

Request attributes are the standard way to pass objects between Servlet and JSP within a forward chain — they exist for the lifetime of a single request.

Passing a List and reading it in JSP

```
// --- Servlet side ---
List<String> errors = new ArrayList<>();
if (name.isEmpty()) errors.add("Name is required.");
if (age < 18)       errors.add("Age must be 18 or above.");

req.setAttribute("errors", errors);
req.setAttribute("formData", req.getParameterMap());

req.getRequestDispatcher("/WEB-INF/views/form.jsp").forward(req, res);

// --- JSP side (form.jsp) ---
```

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

<c:if test="${not empty errors}">
    <ul style="color:red;">
        <c:forEach var="err" items="${errors}">
            <li>${err}</li>
        </c:forEach>
    </ul>
</c:if>
```

5.3.5 Post-Redirect-Get (PRG) Pattern

PRG Pattern

The **Post-Redirect-Get** pattern prevents duplicate form submissions (the browser "re-submit" warning on refresh).

1. Client submits a **POST** request to Servlet.
2. Servlet processes the data (insert into DB, etc.).
3. Servlet issues a **sendRedirect()** to a GET endpoint.
4. Browser follows redirect with a **GET** request.
5. Servlet/JSP renders the success/result page.

Now refreshing the page only re-issues the harmless GET, not the POST.

PRG Pattern Implementation

```java
@WebServlet("/addStudent")
public class AddStudentServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {

        // 1. Process the POST (save to DB)
        String name = req.getParameter("name");
        new StudentDAO().save(new Student(name));

        // 2. Store flash message in session
        req.getSession().setAttribute("flashMsg",
            "Student '" + name + "' added successfully!");

        // 3. Redirect to GET (PRG: prevents re-submission on refresh)
        res.sendRedirect(req.getContextPath() + "/students");
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        // 4. Render result page (safe to refresh)
        List<Student> list = new StudentDAO().getAllStudents();
        req.setAttribute("students", list);

        // 5. Show and clear flash message
        String msg = (String) req.getSession().getAttribute("flashMsg");
        if (msg != null) {
            req.setAttribute("flashMsg", msg);
            req.getSession().removeAttribute("flashMsg");
        }
```

```

        req.getRequestDispatcher("/WEB-INF/views/studentList.jsp")
            .forward(req, res);
    }
}
```

5.4 Module 5 Summary

JSP + Servlet Integration — Module 5 Summary

Topic	Key Points
Combining JSP + Servlet	Servlet = Controller; JSP = View; zero scriptlets in JSP
MVC — Model	JavaBean / DAO; holds data and business logic
MVC — View	JSP with EL + JSTL; reads `request` attributes
MVC — Controller	Servlet; reads params, calls model, sets attributes, forwards
WEB-INF placement	JSPs in `WEB-INF/` block direct URL access; enforce MVC
Front Controller	Single Servlet routes all requests; basis of Spring MVC
`forward()`	Server-side; shares request; URL unchanged; Servlet → JSP
`include()`	Embeds fragment output; control returns to caller
`sendRedirect()`	Client-side new request; URL changes; use for PRG pattern
PRG Pattern	POST → process → redirect → GET; prevents resubmission

Module 5 Key Takeaway

The JSP-Servlet MVC pattern is the **foundation of every Java web framework**. Spring MVC, Struts, and Jakarta Faces all implement the same flow: a front controller Servlet receives requests, delegates to service/model classes, and forwards to a template (View) for rendering. Mastering this pattern at the raw Servlet/JSP level makes understanding any Java web framework straightforward.

MODULE 6 — Hibernate Framework

What is Hibernate?

Hibernate is an open-source, lightweight **Object-Relational Mapping (ORM)** framework for Java. It automates the mapping of Java objects (POJOs) to relational database tables, eliminating most hand-written JDBC code. Hibernate implements the **Java Persistence API (JPA)** specification and is the most widely used ORM framework in the Java ecosystem.

Key facts:

- Developed by Gavin King; now maintained by Red Hat.
- Implements JPA — code written using JPA annotations is portable across ORM providers (EclipseLink, OpenJPA, etc.).
- Handles SQL generation, connection pooling, caching, lazy loading, and transaction management automatically.

6.1 Advantages of ORM over JDBC

6.1.1 Limitations of Plain JDBC

- Developer writes repetitive boilerplate: `Connection`, `PreparedStatement`, `ResultSet`, `close()`.
- Manual mapping from `ResultSet` columns to Java object fields for every query — error-prone and tedious.
- Any change to a table schema requires hunting down and rewriting SQL strings scattered across the codebase.
- No built-in caching; every call hits the database.
- No transparent relationship management (joins, cascades, lazy loading).

6.1.2 How ORM / Hibernate Solves These

JDBC vs. Hibernate — Detailed Comparison

Aspect	JDBC	Hibernate / ORM
Boilerplate code	High; manual connection and statement management	Minimal; framework manages all JDBC calls
SQL writing	Developer writes all SQL strings manually	Auto-generated by framework; HQL for custom queries
Object mapping	Manual column → field mapping per `ResultSet`	Automatic via annotations / XML mapping
Schema changes	Requires updating SQL strings throughout codebase	Change entity class; Hibernate regenerates SQL
Relationships	Manual JOIN queries	Automatic via `@OneToMany`, `@ManyToOne`, etc.
Caching	Not provided	First-level (session) and second-level cache built-in
Database portability	SQL dialect is DB-specific	Hibernate dialect abstracts DB differences
Transaction mgmt	Manual `commit()/rollback()`	Declarative via annotations or config
Lazy / Eager loading	Not available	Built-in via `FetchType.LAZY` / `EAGER`
Productivity	Lower (more code per feature)	Higher (focus on domain model)

When to Still Use JDBC

Hibernate is not always the right tool. Use plain JDBC or Spring JDBC when: (a) you need fine-grained control over complex native SQL or stored procedures; (b) performance is critical and ORM overhead is unacceptable (e.g. bulk ETL of millions of rows); (c) the project is very small and adding Hibernate's dependencies is overkill. Many enterprise applications mix both — Hibernate for CRUD, JDBC for performance-critical bulk operations.

6.2 Hibernate Architecture and Core Components

Hibernate's architecture sits between the application and the database, intercepting all persistence operations.

6.2.1 Layered Architecture Overview

Hibernate Layered Architecture

Layer	Description
Java Application	Your POJOs, DAO classes, business logic
Hibernate API	`SessionFactory`, `Session`, `Transaction`, `Query`
JDBC / JTA	Hibernate internally uses JDBC for SQL; JTA for distributed transactions
Database	MySQL, PostgreSQL, Oracle, H2, etc. (abstracted by dialect)

6.2.2 Core Components

Hibernate Core Components — Detailed

1. Configuration (`org.hibernate.cfg.Configuration`)
Reads `hibernate.cfg.xml` or annotation metadata. Bootstraps the `SessionFactory`. Done *once* at application startup.

2. SessionFactory (`org.hibernate.SessionFactory`)
A **heavyweight**, thread-safe factory for `Session` objects. One `SessionFactory` per database per application. Expensive to create; created once and reused. Internally maintains the connection pool and second-level cache.

3. Session (`org.hibernate.Session`)
A **lightweight**, single-threaded, short-lived object representing a unit of work with the database. Analogous to a JDBC `Connection`. Obtained from `SessionFactory.openSession()`. Provides `save()`, `get()`, `update()`, `delete()`, and `createQuery()`.

4. Transaction (`org.hibernate.Transaction`)
Wraps the database transaction. Obtained from `session.beginTransaction()`. Must be explicitly committed or rolled back.

5. Query (`org.hibernate.Query` / `org.hibernate.query.Query`)
Used to execute HQL (Hibernate Query Language) or native SQL. Supports named queries, pagination, and parameter binding.

6. Criteria (`CriteriaBuilder` / `CriteriaQuery`)
Type-safe, programmatic API for building queries without writing query strings. Part of

JPA 2.0+.

7. Persistent Object (Entity)
A Java POJO whose instances are managed by Hibernate. Must have a no-arg constructor and a primary key field. Can exist in three states (see below).

6.2.3 Object / Entity States in Hibernate

Every entity object in Hibernate is always in one of three states:

Hibernate Object States

State	Description	Managed by Session?
Transient	Object created with `new`; not associated with any Session; no DB row	No
Persistent	Associated with an open Session; changes are automatically synchronised to DB at flush/commit	Yes
Detached	Was persistent; Session has been closed; changes are NOT tracked	No (was)

Entity State Transitions

```java
Student s = new Student("Alice", 3.9); // Transient: no DB row

Session session = sessionFactory.openSession();
session.beginTransaction();

session.save(s);    // Transient -> Persistent: INSERT queued

session.getTransaction().commit();
session.close();    // Persistent -> Detached: session closed

// s is now Detached: changes not tracked
s.setGpa(4.0);

Session session2 = sessionFactory.openSession();
session2.beginTransaction();
session2.update(s); // Detached -> Persistent again: UPDATE queued
session2.getTransaction().commit();
session2.close();
```

6.2.4 First-Level Cache (Session Cache)

Hibernate's **first-level cache** is enabled by default and is scoped to a `Session`. Within the same session, if you call `get()` for the same entity twice, Hibernate issues only *one* SQL query; the second call returns the cached object. The cache is cleared when the session is closed.

First-Level Cache in Action

```java
Session session = sessionFactory.openSession();

Student s1 = session.get(Student.class, 101); // SQL SELECT fired
Student s2 = session.get(Student.class, 101); // Returns from cache (no SQL)

System.out.println(s1 == s2); // true -- same object from cache

session.close();
```

6.3 Setting Up Hibernate in a Java Application

6.3.1 Project Dependencies (Maven)

pom.xml -- Hibernate + MySQL Dependencies

```xml
<dependencies>

    <!-- Hibernate Core -->
    <dependency>
        <groupId>org.hibernate</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>5.6.15.Final</version>
    </dependency>

    <!-- MySQL JDBC Driver -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>

    <!-- JPA API (included in hibernate-core but explicit for clarity) -->
    <dependency>
        <groupId>javax.persistence</groupId>
        <artifactId>javax.persistence-api</artifactId>
        <version>2.2</version>
    </dependency>

</dependencies>
```

6.3.2 HibernateUtil — SessionFactory Helper

It is best practice to create a utility class that initialises the `SessionFactory` once and provides it application-wide.

HibernateUtil.java

```java
package com.example.util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

    private static final SessionFactory sessionFactory;

    static {
        try {
            // Reads hibernate.cfg.xml from classpath
            sessionFactory = new Configuration()
                                .configure()
                                .buildSessionFactory();
        } catch (Exception e) {
            throw new ExceptionInInitializerError(
                "SessionFactory creation failed: " + e);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }

    public static void shutdown() {
        getSessionFactory().close();
    }
}
```

6.4 Hibernate Configuration

Hibernate can be configured either via **XML** (`hibernate.cfg.xml`) or via **JPA annotations** on entity classes. Both approaches can be mixed.

6.4.1 XML Configuration — `hibernate.cfg.xml`

This file is placed in `src/main/resources/` (Maven) or the classpath root.

hibernate.cfg.xml -- Complete Configuration

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
  <session-factory>

    <!-- Database Connection -->
    <property name="hibernate.connection.driver_class">
        com.mysql.cj.jdbc.Driver
    </property>
    <property name="hibernate.connection.url">
        jdbc:mysql://localhost:3306/hibernatedb?useSSL=false
    </property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">password</property>

    <!-- Hibernate Dialect -->
    <!-- Tells Hibernate which SQL flavour to generate -->
    <property name="hibernate.dialect">
        org.hibernate.dialect.MySQL8Dialect
    </property>

    <!-- Connection Pool (built-in C3P0) -->
    <property name="hibernate.c3p0.min_size">5</property>
    <property name="hibernate.c3p0.max_size">20</property>
    <property name="hibernate.c3p0.timeout">300</property>

    <!-- Schema Auto-Management -->
    <!-- validate | update | create | create-drop -->
    <property name="hibernate.hbm2ddl.auto">update</property>

    <!-- Logging -->
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>

    <!-- Registered Entity Classes -->
    <mapping class="com.example.model.Student" />
    <mapping class="com.example.model.Course" />

  </session-factory>
</hibernate-configuration>
```

hbm2ddl.auto Values

Value	Behaviour
`validate`	Validates schema against entities; throws error if mismatch; no changes
`update`	Alters existing tables to match entities; safe for dev/prod
`create`	Drops and recreates all tables on startup; **all data lost**
`create-drop`	Creates tables on startup; drops them on SessionFactory close

6.5 Mapping Java Classes to Database Tables

6.5.1 Annotation-Based Mapping (Recommended)

JPA annotations placed directly on the entity class replace the need for a separate `.hbm.xml` mapping file. This is the modern, recommended approach.

Student.java -- Full Entity with Annotations

```java
package com.example.model;

import javax.persistence.*;

@Entity                          // Marks this class as a Hibernate entity
@Table(name = "students")        // Maps to the "students" table
public class Student {

    @Id                                     // Primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT
    @Column(name = "id")
    private int id;

    @Column(name = "name", nullable = false, length = 100)
    private String name;

    @Column(name = "roll_no", unique = true)
    private int rollNo;

    @Column(name = "gpa", precision = 3, scale = 2)
    private double gpa;

    @Column(name = "email", unique = true, length = 150)
    private String email;

    @Transient                              // Not persisted to DB
    private String tempToken;
```

```java

    public Student() {}                    // Required no-arg constructor

    public Student(String name, int rollNo, double gpa, String email) {
        this.name   = name;
        this.rollNo = rollNo;
        this.gpa    = gpa;
        this.email  = email;
    }

    // Getters and setters
    public int    getId()      { return id;     }
    public String getName()    { return name;   }
    public int    getRollNo()  { return rollNo; }
    public double getGpa()     { return gpa;    }
    public String getEmail()   { return email;  }

    public void setId(int id)          { this.id = id;         }
    public void setName(String name)   { this.name = name;     }
    public void setRollNo(int r)       { this.rollNo = r;       }
    public void setGpa(double gpa)     { this.gpa = gpa;       }
    public void setEmail(String email) { this.email = email;   }
}
```

Key JPA / Hibernate Mapping Annotations

Annotation	Purpose
`@Entity`	Declares the class as a persistent entity
`@Table(name=...)`	Specifies the mapped table name
`@Id`	Marks the primary key field
`@GeneratedValue`	Strategy for PK generation (IDENTITY, SEQUENCE, AUTO, TABLE)
`@Column`	Maps field to column; set name, nullable, unique, length
`@Transient`	Field is NOT persisted
`@Temporal`	For `java.util.Date`: `DATE`, `TIME`, `TIMESTAMP`
`@Lob`	Maps to BLOB/CLOB for large binary or text data
`@OneToOne`	One-to-one relationship
`@OneToMany`	One-to-many relationship (parent side)
`@ManyToOne`	Many-to-one relationship (child side)
`@ManyToMany`	Many-to-many relationship
`@JoinColumn`	Specifies foreign key column
`@Enumerated`	Maps Java enum to DB (ORDINAL or STRING)

6.5.2 XML-Based Mapping — `Student.hbm.xml` (Legacy)

Before annotations, a separate HBM XML file mapped each class to its table.

Student.hbm.xml -- XML Mapping File

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
  <class name="com.example.model.Student" table="students">

    <id name="id" column="id">
      <generator class="identity" />
    </id>
```

```

    <property name="name"   column="name"    not-null="true" length="100" />
    <property name="rollNo" column="roll_no" unique="true" />
    <property name="gpa"    column="gpa" />
    <property name="email"  column="email"   unique="true" />

  </class>
</hibernate-mapping>
```

Annotations vs. HBM XML

Annotations are preferred in all modern projects — mapping lives with the class, reducing file proliferation and making refactoring easier. Use HBM XML only when you cannot modify the source class (e.g. mapping legacy third-party classes) or when mapping must be changed without recompiling.

6.6 Basic CRUD Operations

All CRUD operations follow the same pattern:

1. Obtain a `Session` from `HibernateUtil.getSessionFactory()`.
2. Begin a `Transaction`.
3. Perform the operation.
4. Commit the transaction (or rollback on exception).
5. Close the session.

6.6.1 Save (Create)

`session.save(entity)` persists a transient object, issuing an `INSERT`. Returns the generated primary key.

`session.persist(entity)` is the JPA-standard equivalent (void return).

Save / Create

```java
public static void saveStudent(Student student) {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Transaction tx  = null;
    try {
        tx = session.beginTransaction();

        session.save(student);      // INSERT INTO students ...
        // OR: session.persist(student); // JPA-standard, void

        tx.commit();
        System.out.println("Saved with ID: " + student.getId());

```

```
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
}

// Usage
Student s = new Student("Alice", 101, 3.9, "alice@example.com");
saveStudent(s);
```

6.6.2 Retrieve (Read)

Hibernate provides three main ways to retrieve a single entity:

get() vs. load() vs. find()

Method	Behaviour
`session.get(Class, id)`	Hits the DB immediately; returns `null` if not found
`session.load(Class, id)`	Returns a proxy (lazy); throws `ObjectNotFoundException` if not found when accessed
`session.find(Class, id)`	JPA equivalent of `get()`; preferred with JPA

Retrieve by Primary Key

```
public static Student getStudent(int id) {
    Session session = HibernateUtil.getSessionFactory().openSession();
    try {
        Student s = session.get(Student.class, id);
        if (s != null) {
            System.out.println("Found: " + s.getName() + " GPA: " + s.getGpa
                ());
        } else {
            System.out.println("No student with id " + id);
        }
        return s;
    } finally {
        session.close();
    }
}
```

Retrieve All Records

```java
public static List<Student> getAllStudents() {
    Session session = HibernateUtil.getSessionFactory().openSession();
    try {
        // HQL query -- uses class name, not table name
        List<Student> students =
            session.createQuery("FROM Student", Student.class).list();

        students.forEach(s ->
            System.out.println(s.getId() + " | " +
                               s.getName() + " | " + s.getGpa()));
        return students;
    } finally {
        session.close();
    }
}
```

6.6.3 Update

`session.update(entity)` re-attaches a detached object and issues an `UPDATE`. `session.merge(entity)` is the JPA-standard equivalent that also works if the entity may or may not be managed.

Update

```java
public static void updateStudent(int id, double newGpa) {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Transaction tx  = null;
    try {
        tx = session.beginTransaction();

        Student s = session.get(Student.class, id);
        if (s != null) {
            s.setGpa(newGpa);        // Change the persistent object
            // No explicit update call needed while session is open!
            // Hibernate detects the change at flush/commit (dirty checking).
        }

        tx.commit();     // UPDATE issued here via dirty checking
        System.out.println("Updated GPA for student " + id);

    } catch (Exception e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
}
```

Update a Detached Entity with session.update()

```java
// Entity obtained in a previous session (detached)
Student detachedStudent = getStudent(101);
detachedStudent.setEmail("newemail@example.com");

// Re-attach and update in a new session
Session session = HibernateUtil.getSessionFactory().openSession();
Transaction tx  = session.beginTransaction();

session.update(detachedStudent);   // Detached -> Persistent; UPDATE queued

tx.commit();
session.close();
```

Automatic Dirty Checking

When an entity is in the **Persistent** state (inside an open Session), you do *not* need to call `session.update()`. Hibernate automatically detects changes to managed objects (**dirty checking**) and synchronises them to the DB at `flush()` time (typically just before `commit()`). Explicit `update()` is only needed for **detached** entities.

6.6.4 Delete

`session.delete(entity)` issues a `DELETE` statement. The entity must be in the Persistent or Detached state.

Delete

```java
public static void deleteStudent(int id) {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Transaction tx  = null;
    try {
        tx = session.beginTransaction();

        Student s = session.get(Student.class, id);
        if (s != null) {
            session.delete(s);     // DELETE FROM students WHERE id = ?
            System.out.println("Deleted student with id " + id);
        } else {
            System.out.println("Student not found.");
        }

        tx.commit();

    } catch (Exception e) {
        if (tx != null) tx.rollback();
```

```
        e.printStackTrace();
    } finally {
        session.close();
    }
}
```

6.6.5 saveOrUpdate() and merge()

Useful Combined Methods

Method	Behaviour
`session.saveOrUpdate(entity)`	Calls `save()` if new (id is 0/null); `update()` otherwise. Hibernate-specific.
`session.merge(entity)`	JPA-standard; copies state of detached entity to a managed copy and returns it.

saveOrUpdate() Example

```
Session session = HibernateUtil.getSessionFactory().openSession();
Transaction tx  = session.beginTransaction();

Student s = new Student("Bob", 102, 3.5, "bob@example.com");
s.setId(0);                 // id = 0 -> save() (INSERT)
session.saveOrUpdate(s);

Student existing = session.get(Student.class, 101);
existing.setGpa(3.8);
session.saveOrUpdate(existing); // id != 0 -> update() (UPDATE)

tx.commit();
session.close();
```

6.7 Hibernate Query Language (HQL)

What is HQL?

HQL (Hibernate Query Language) is Hibernate's object-oriented query language. It is syntactically similar to SQL but operates on **entity classes and their properties** rather than table names and column names. Hibernate translates HQL into the appropriate SQL dialect at runtime, making HQL queries *database-independent*.

HQL vs. SQL

Aspect	SQL	HQL
Operates on	Table names, column names	Entity class names, field names
Case sensitivity	Case-insensitive (table/col)	Class names case-sensitive; keywords case-insensitive
DB portability	DB-specific dialect	DB-independent; Hibernate translates
Joins	SQL JOIN syntax	HQL association traversal (`s.course.name`)
Return type	Raw rows	Typed Java objects / lists

6.7.1 Basic HQL Syntax

HQL Query Examples

```java
Session session = HibernateUtil.getSessionFactory().openSession();

// 1. Select all -- note: class name "Student", not table "students"
List<Student> all = session.createQuery("FROM Student", Student.class)
                            .list();

// 2. Select with WHERE clause -- use field name "name", not column "name"
List<Student> result = session.createQuery(
    "FROM Student WHERE gpa > 3.5", Student.class).list();

// 3. Order by
List<Student> ordered = session.createQuery(
    "FROM Student ORDER BY gpa DESC", Student.class).list();

// 4. Select specific columns (returns Object[])
List<Object[]> cols = session.createQuery(
    "SELECT s.name, s.gpa FROM Student s WHERE s.gpa >= 3.8").list();
for (Object[] row : cols) {
    System.out.println(row[0] + " -- " + row[1]);
}

session.close();
```

6.7.2 Named Parameters in HQL

Always use named parameters (never string concatenation) to avoid SQL injection.

Named Parameters

```java
Session session = HibernateUtil.getSessionFactory().openSession();

// Named parameter with :paramName
String hql = "FROM Student s WHERE s.name = :studentName AND s.gpa > :minGpa"
    ;

List<Student> result = session.createQuery(hql, Student.class)
    .setParameter("studentName", "Alice")
    .setParameter("minGpa", 3.5)
    .list();

result.forEach(s -> System.out.println(s.getName() + " | " + s.getGpa()));

session.close();
```

6.7.3 Pagination with HQL

Pagination -- setFirstResult and setMaxResults

```java
Session session = HibernateUtil.getSessionFactory().openSession();

int pageNumber = 2;   // 1-based page number
int pageSize   = 5;   // records per page

List<Student> page = session.createQuery("FROM Student ORDER BY id", Student.
    class)
    .setFirstResult((pageNumber - 1) * pageSize)  // offset
    .setMaxResults(pageSize)                       // limit
    .list();

page.forEach(s -> System.out.println(s.getId() + " | " + s.getName()));

session.close();
```

6.7.4 Aggregate Functions in HQL

HQL supports all standard aggregate functions: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

Aggregate Functions

```java
Session session = HibernateUtil.getSessionFactory().openSession();

// Count all students
long count = (long) session.createQuery("SELECT COUNT(s) FROM Student s")
                           .uniqueResult();
System.out.println("Total: " + count);
```

```java

// Average GPA
double avgGpa = (double) session.createQuery(
    "SELECT AVG(s.gpa) FROM Student s").uniqueResult();
System.out.println("Avg GPA: " + avgGpa);

// Max GPA
double maxGpa = (double) session.createQuery(
    "SELECT MAX(s.gpa) FROM Student s").uniqueResult();
System.out.println("Max GPA: " + maxGpa);

session.close();
```

6.7.5 HQL UPDATE and DELETE (Bulk Operations)

HQL can also perform bulk `UPDATE` and `DELETE` without loading entities into memory — useful for mass operations.

Bulk UPDATE and DELETE in HQL

```java
Session session = HibernateUtil.getSessionFactory().openSession();
Transaction tx  = session.beginTransaction();

// Bulk UPDATE: boost GPA by 0.1 for all CS students
int updatedRows = session.createQuery(
    "UPDATE Student s SET s.gpa = s.gpa + 0.1 WHERE s.gpa < 3.5")
    .executeUpdate();
System.out.println("Updated rows: " + updatedRows);

// Bulk DELETE: remove students with GPA below 2.0
int deletedRows = session.createQuery(
    "DELETE FROM Student s WHERE s.gpa < 2.0")
    .executeUpdate();
System.out.println("Deleted rows: " + deletedRows);

tx.commit();
session.close();
```

Bulk Operations Bypass First-Level Cache

Bulk HQL `UPDATE` and `DELETE` execute directly in the database and **bypass the first-level cache**. Any entities loaded in the same session before the bulk operation reflect stale data. Always execute bulk operations at the start of a session or call `session.clear()` afterwards.

6.7.6 Named Queries

Named queries are pre-defined HQL queries declared on the entity class with `@NamedQuery`. They are validated at startup and reusable throughout the application.

Named Query Declaration and Usage

```java
// On the entity class:
@Entity
@Table(name = "students")
@NamedQuery(
    name  = "Student.findByGpaAbove",
    query = "FROM Student s WHERE s.gpa >= :minGpa ORDER BY s.gpa DESC"
)
@NamedQuery(   // Multiple named queries via @NamedQueries
    name  = "Student.findAll",
    query = "FROM Student ORDER BY s.name"
)
public class Student { /* ... */ }

// Using a named query:
Session session = HibernateUtil.getSessionFactory().openSession();

List<Student> topStudents = session
    .createNamedQuery("Student.findByGpaAbove", Student.class)
    .setParameter("minGpa", 3.7)
    .list();

topStudents.forEach(s -> System.out.println(s.getName()));
session.close();
```

6.7.7 Native SQL Queries

When HQL is insufficient (e.g. complex database-specific queries), you can execute native SQL through Hibernate.

Native SQL Query

```java
Session session = HibernateUtil.getSessionFactory().openSession();

// Returns a list of Student entities from native SQL
List<Student> result = session
    .createNativeQuery("SELECT * FROM students WHERE gpa > 3.5",
                       Student.class)
    .list();

// Returns raw Object[]
List<Object[]> raw = session
```

```
    .createNativeQuery("SELECT name, gpa FROM students ORDER BY gpa DESC")
    .list();

for (Object[] row : raw) {
    System.out.println(row[0] + " | " + row[1]);
}

session.close();
```

6.8 Module 6 Summary

Hibernate Complete Reference — Module 6 Summary

Topic	Key Points
ORM vs. JDBC	ORM eliminates boilerplate; auto-generates SQL; handles relationships, caching
Hibernate overview	Open-source ORM; implements JPA; DB-independent via dialects
Architecture	Configuration → SessionFactory (one) → Session (per request) → Transaction
Object states	Transient (new) → Persistent (in session) → Detached (session closed)
1st-level cache	Scoped to Session; same id returns cached object; enabled by default
Configuration	`hibernate.cfg.xml`: connection, dialect, `hbm2ddl.auto`, show_sql
Annotations	`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@Transient`
Save	`session.save()` / `persist()` → INSERT
Retrieve	`session.get()` (immediate) / `load()` (proxy/lazy)
Update	Dirty checking (automatic in session) / `session.update()` for detached
Delete	`session.delete(entity)` → DELETE

saveOrUpdate	INSERT if new; UPDATE if existing; based on id value
HQL basics	Uses class/field names; DB-independent; translated to SQL by Hibernate
HQL parameters	Always use `:namedParam`; never string concatenation
Pagination	`setFirstResult(offset).setMaxResults(limit)`
Aggregates	`COUNT, SUM, AVG, MIN, MAX` supported in HQL
Bulk HQL	`executeUpdate()` for mass UPDATE/DELETE; bypasses cache
Named queries	`@NamedQuery` on entity; validated at startup; reusable
Native SQL	`createNativeQuery()` for DB-specific queries

Module 6 Key Takeaway

Hibernate transforms database interaction from a low-level, error-prone process (manual JDBC) into a high-level, object-oriented one. The three pillars to master are: (1) the **Session/Transaction lifecycle** — always open, use, and close correctly; (2) **entity state management** — understand transient, persistent, and detached to avoid stale-data bugs; and (3) **HQL** — write portable, injection-safe queries that operate on your domain model. With these three in hand, you have a solid foundation for every higher-level framework — Spring Data JPA, Micronaut Data, Quarkus Panache — that sits on top of Hibernate.